DATA INGESTION

KUBERNETES, OPENSHIFT E OPENSTACK

5

PÚBLICA

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE FIGURAS

Figura 1 – Representação do Kubernetes 6
Figura 2 – Componentes da Arquitetura Kubernetes 7
Figura 3 – Funcionamento do Kubernetes 9
Figura 4 – Kubernetes Control Plane com Linux e Windows 10
Figura 5 – Representação do Red Hat OpenShift do Azure 12
Figura 6 – Componentes da Arquitetura do OpenShift 13
Figura 7 – Ferramentas do OpenShift 15
Figura 8 – Representação do OpenStack 16
Figura 9 – OpenStack Landscape 17
Figura 10 – Pilares da infraestrutura OpenStack 19
Figura 11 – Tela inicial do VirtualBox 20
Figura 12 – Criando uma VM nova no VirtualBox 21
Figura 13 – Configurando a quantidade de ram da VM no VirtualBox 22
Figura 14 – Configurando a quantidade de espaço em disco da VM no VirtualBox 23
Figura 15 – Configurando a interface de rede da VMno VirtualBox 24
Figura 16 – Tela de boas-vindas do OpenStack 26
Figura 17 – Papel do Kubernetes e do Docker Swarm 27
Figura 18 – Relação do Kubernetes e Microsserviços 28
Figura 19 - Kubernetes Service 30
Figura 20 - Create Kubernetes Service Cluster 31
Figura 21 - Configurações Básicas 32
Figura 22 - Configuração dos nodes 33
Figura 23 - Revisar e Criar 34
Figura 24 – Criar 35
Figura 25 - Implantação do Cluster 36
Figura 26 - Go to Resource 36
Figura 27 – Criando uma aplicação (1) 37
Figura 28 - Criando uma aplicação (2) 38
Figura 29 - Criando uma aplicação (3) 39
Figura 30 - Criando uma aplicação (4) 40
Figura 31 - Acessando a aplicação 41
Figura 32 - Aplicação Web 42
Figura 33 - Removendo o Cluster Kubernetes 42
Figura 34 - Removendo o Cluster Kubernetes(2) 43

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE QUADROS

Quadro 1 – Componentes e serviços do OpenStack 18

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

SUMÁRIO

1 KUBERNETES, OPENSHIFT E OPENSTACK 5
1.1 Kubernetes 5
1.1.1 Arquitetura do Kubernetes 7
1.1.2 Funcionamento do Kubernetes 9
2 OPENSHIFT E AZURE RED HAT OPEN SHIFT 11
2.1 Arquitetura do OpenShift 13
2.2 Funcionamento do OpenShift 14
3 OPENSTACK 16
3.1 Arquitetura do OpenStack 17
3.2 Funcionamento do OpenStack 18
3.3 Instalando o DevStack 19
4 ENTENDENDO ALGUMAS COMPARAÇÕES 27
4.1 Kubernetes e Docker 27
4.2 Kubernetes e Microsserviços 28
4.3 Kubernetes x OpenShift x OpenStack 28
5 HANDS-ON – IMPLEMENTAÇÃO E CONFIGURAÇÃO DO KUBERNETES USANDO O PORTAL AZURE 30
CONCLUSÃO 45
REFERÊNCIAS 46
LISTA DE ABREVIATURAS 47

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1 KUBERNETES, OPENSHIFT E OPENSTACK

Para gerir o complexo funcionamento dos diversos microsserviços de uma aplicação, existem **gestores de containers** e **gestores de recursos**.

Os gestores de containers são ferramentas responsáveis por gerir o que está em execução em cada cluster, viabilizando a escalabilidade e a disponibilidade dos serviços. Nesse cenário, vamos falar sobre o Kubernetes.

Os gestores de recursos são ferramentas utilizadas para orquestrar os componentes da aplicação, como um mecanismo de armazenamento de banco de dados, um serviço de registro em log, um servidor de integração contínua, um servidor web, componentes em nuvem, entre outros. Nesse cenário, vamos falar sobre o OpenShift e o OpenStack.

1.1 Kubernetes

Kubernetes é uma plataforma de código aberto que foi criada e desenvolvida pelos engenheiros da Google para **orquestrar** e **gerenciar um *cluster* de containers** Linux como um único sistema. Suas principais características são:

- Organização em *clusters*.
- Automatização de execução de containers.
- Replicação de containers.
- Autoescalonamento dos containers.
- Suporte a containers Docker.
- Suporte a volumes persistentes remotos.
- Capacidade de integração com serviços de segurança, rede, armazenamento, monitoramento, medição e outros.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

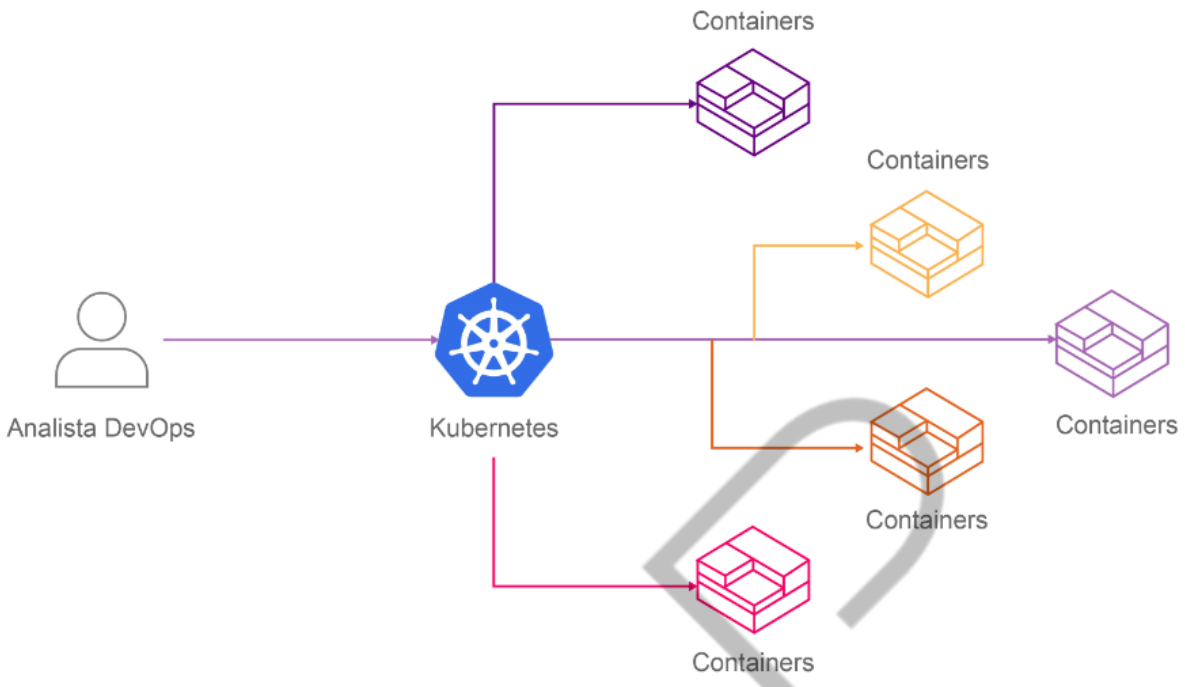

Figura 1 – Representação do Kubernetes
Fonte: adaptado por FIAP (2019)

Dentre as funcionalidades desempenhadas pelo Kubernetes, temos:

- Gerenciamento e automatização de grande parte das implantações e atualizações de aplicativos.
- Otimização do uso do hardware, aumentando a disponibilidade dos recursos para executar os aplicativos.
- Capacidade de orquestrar containers em *clouds* privadas, públicas ou híbridas e também em múltiplos *hosts*.
- Garantia de integridade e asseguração de que os aplicativos sejam autorrecuperáveis em seus containers, com reinício, posicionamento, escalonamento automáticos e replicação.
- Maior agilidade para escalar aplicativos em containers e todos os recursos relacionados.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

1.1.1 Arquitetura do Kubernetes

Vamos entender alguns conceitos importantes e termos específicos que permeiam o funcionamento do Kubernetes. Como você pode observar na Figura "Componentes da Arquitetura Kubernetes", a arquitetura do Kubernetes possui um servidor mestre (**Kubernetes Master)**, um servidor de node (**Kubernetes Node)**; e uma interface de comunicação (**Kubectl)** entre DevOps e servidor.

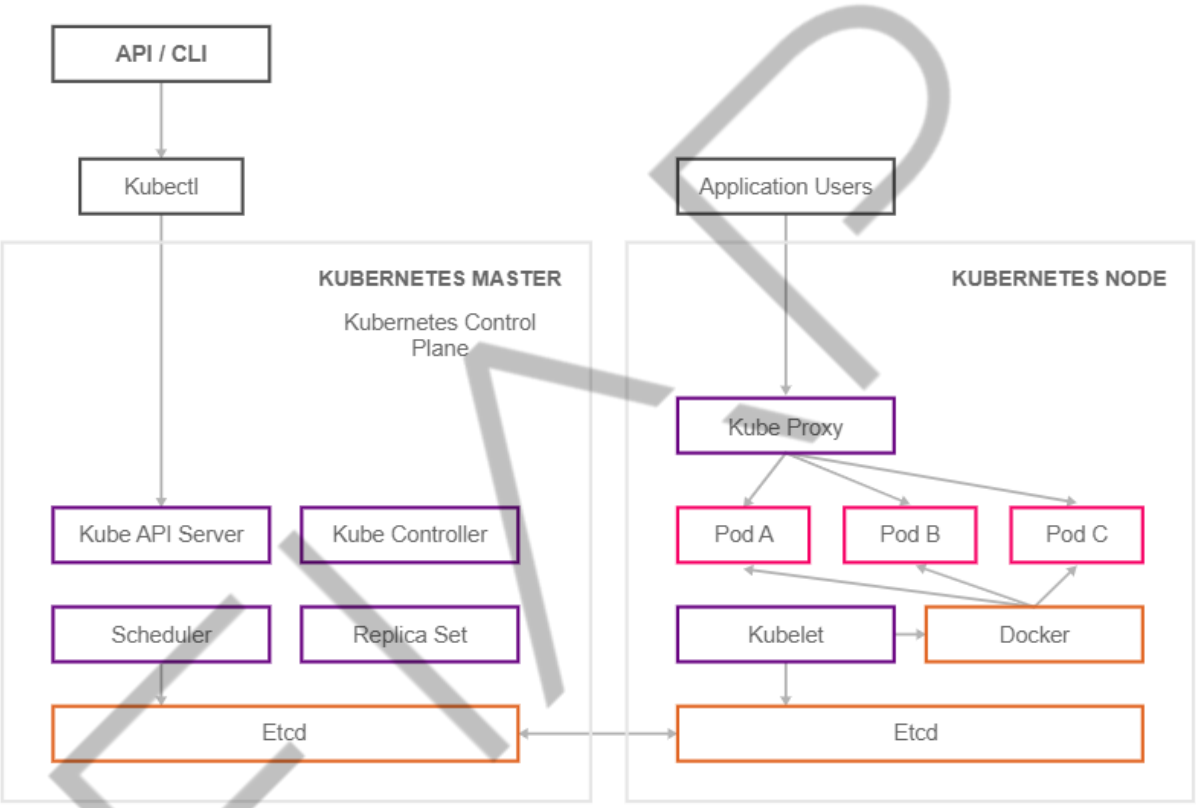

Figura 2 – Componentes da Arquitetura Kubernetes
Fonte: adaptado por FIAP (2019)

Interface de comunicação – Kubectl

- **Kubectl:** é uma interface de linha de comando que se comunica com um *cluster* Kubernetes por meio de uma API a partir de um computador local (sob comando de um DevOps).

Componentes do Servidor Mestre

- **Etcd:** é usado como armazenamento de backup do Kubernetes. É uma estrutura de armazenamento baseada em chave/valor e pode ser configurada e disponibilizada de forma global para vários *nodes*.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Kube API Server**: responsável por expor a API do Kubernetes por meio do uso do JSON sobre HTTP.
- **Scheduler:** realiza um trabalho de gerenciamento de *pod* recém-criado para designar em qual nó deve ser executado, baseando-se na disponibilidade de recursos.
- **Kube Controller ou Controller Manager:** é um processo que executa os controladores, como o Replication Controller, Node Controller, Endpoints Controller e o Service Account & Token Controllers. Cada um desses controladores é um processo, porém todos são compilados em um único binário e executados em um único processo.
- **ReplicaSet**: responsável por garantir a disponibilidade, a qualquer momento, de um número específico de réplicas de *pods*. O ReplicaSet veio como alternativa ao ReplicationController.
- **ReplicationController**: desempenha as mesmas funcionalidades e comportamentos do ReplicaSet, com exceção de que o ReplicationController não oferece suporte aos requisitos de seletor de rótulos baseado em conjuntos. Por esse motivo, o ReplicaSet é o preferido para a função.

Componentes do Servidor de Node

- **Kubernetes Node**: é uma VM (*virtual machine*) ou uma máquina física em um determinado *cluster,* controlada no Kubernetes. É responsável pela execução dos seus aplicativos, que são agrupados pelos *pods*. Um nó contém os serviços Docker, Kubelet e Kube-Proxy.
- **Kubelet**: é um pequeno serviço executado em cada nó no *cluster* e comunica-se com os componentes do servidor mestre para autenticar e receber comandos e trabalho.
- **Kube-Proxy**: como o próprio nome diz, esse componente é um proxy de rede responsável por encaminhar solicitações e, assim como o Kubelet, também é executado em cada nó no *cluster.*

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Docker**: responsável por rodar e executar os containers sob comando do Kubernetes.
- **Pod**: é um objeto simples que encapsula um ou mais containers controlados por uma única aplicação.

1.1.2 Funcionamento do Kubernetes

O Kubernetes baseia seu funcionamento em camadas: a camada mais alta desempenha um papel de *gateway* para acesso ao container e os níveis mais baixos, denominados *pods*, criam uma camada extra de abstração que contém toda a complexidade necessária para trabalhar, por exemplo, com rede, armazenamento compartilhado, entre outros serviços.

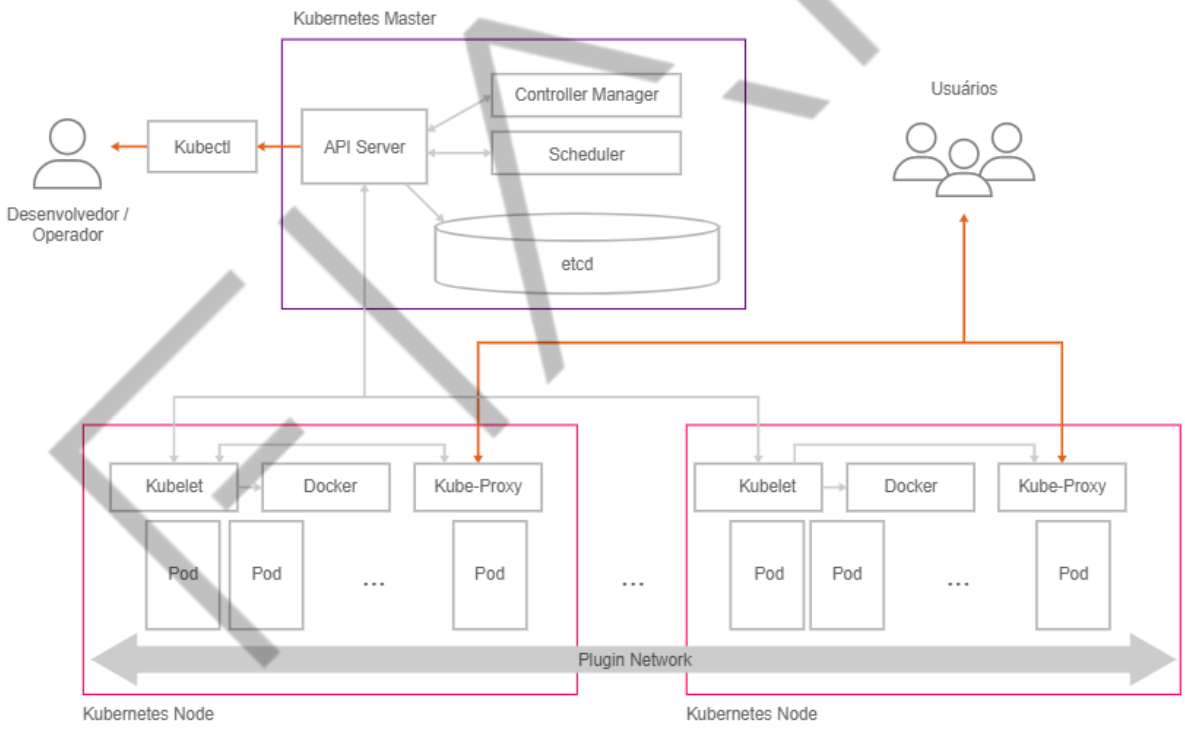

Figura 3 – Funcionamento do Kubernetes
Fonte: adaptado por FIAP (2019)

Os DevOps interagem com a camada mais alta por meio da comunicação com a API Server, a principal API do servidor. O usuário final interage com os *pods,* por meio de *clients* e bibliotecas, os Kube-Proxy.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

É necessário haver um plano declarativo de controle (*Kubernetes Control Plane*), em formato JSON ou YAML, para definir o que deve ser criado e como deve ser gerenciado.

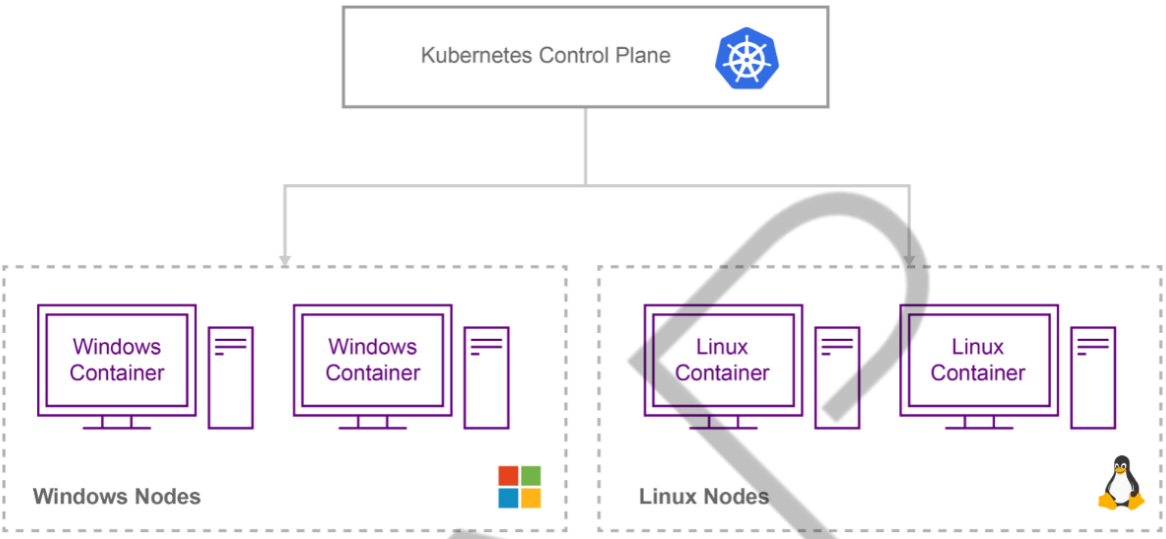

Figura 4 – Kubernetes Control Plane com Linux e Windows
Fonte: adaptado por FIAP (2019)

O plano de controle é utilizado pelo servidor mestre para identificar qual é o estado atual do sistema e quais são os requisitos para executar o aplicativo ou serviço na infraestrutura (dentro do *pod*). Como você viu, os *pods*, que contêm o aplicativo ou serviço que foi definido pelo usuário, representam a última camada do Kubernetes.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2 OPENSHIFT E AZURE RED HAT OPEN SHIFT

O **OpenShift** tem grande adesão no mundo corporativo porque oferece **componentes além do Kubernetes**, ofertando-os como projeto encapsulado. Ele surgiu da necessidade de trabalhar com **mecanismos de controle** e **processos mais burocráticos**, deficientes no Kubernetes.

Ele tem como características importantes o fato de ser inerentemente um **sistema distribuído**, o que beneficia o desenvolvimento e a implementação de aplicativos na nuvem. Além disso, ele é **fundamentado na arquitetura baseada em microsserviços**, que é muito importante para prover funcionalidades diversas, de acordo com a necessidade de que cada projeto.

O **OpenShift** é uma **plataforma de código aberto** relacionada à nuvem que usa a tecnologia de **conteinerização**, seguindo os principais conceitos do Kubernetes. Além da **orquestração de containers** de forma independente da plataforma na qual os containers são executados, o OpenShift auxilia no processo de **gerenciamento de containers** e também disponibiliza uma **interface do usuário** baseada na web. Ele oferece **Plataforma como Serviço** (*PaaS – Platform as a Service*) e disponibiliza **integração facilitada com outras ferramentas e SDKs** (*Software Development Kit*) para diferentes linguagens. Dentre a infinidade de serviços oferecidos pelo OpenShift, temos:

- Recurso de CI / CD (*Continuos Integration / Continous Delivery*) integrados com Jenkins, solução de software universal, genérica e madura que disponibiliza, por exemplo, autenticação OAuth e diversos outros recursos importantes que tornam todo o CI / CD mais “leves”.
- Políticas-padrão de segurança rígidas, tendo como parte integrante o Controle de Acesso Baseado em Função (*RBAC – Role Based Access Control*) para restringir o acesso a usuários autorizados.
- *Health check* das aplicações e containers que estão sendo gerenciados.
- Permição para a executar serviços de nuvem, como Google Compute Engine, AWS, entre outros.
- Gerenciamento de configurações e *logs*.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- Facilidades de implantação e testes, excluindo a necessidade de manuseio de servidores físicos ou virtuais.

A plataforma foi criada pela Red Hat Inc, que firmou parceria com a Microsoft, em maio de 2019, para oferecer pela primeira vez ao mercado o serviço **Azure Red Hat Open Shift.** A novidade dispõe de uma *cloud* híbrida, nesse caso, uma *cloud* pública operando ao lado da virtualização.

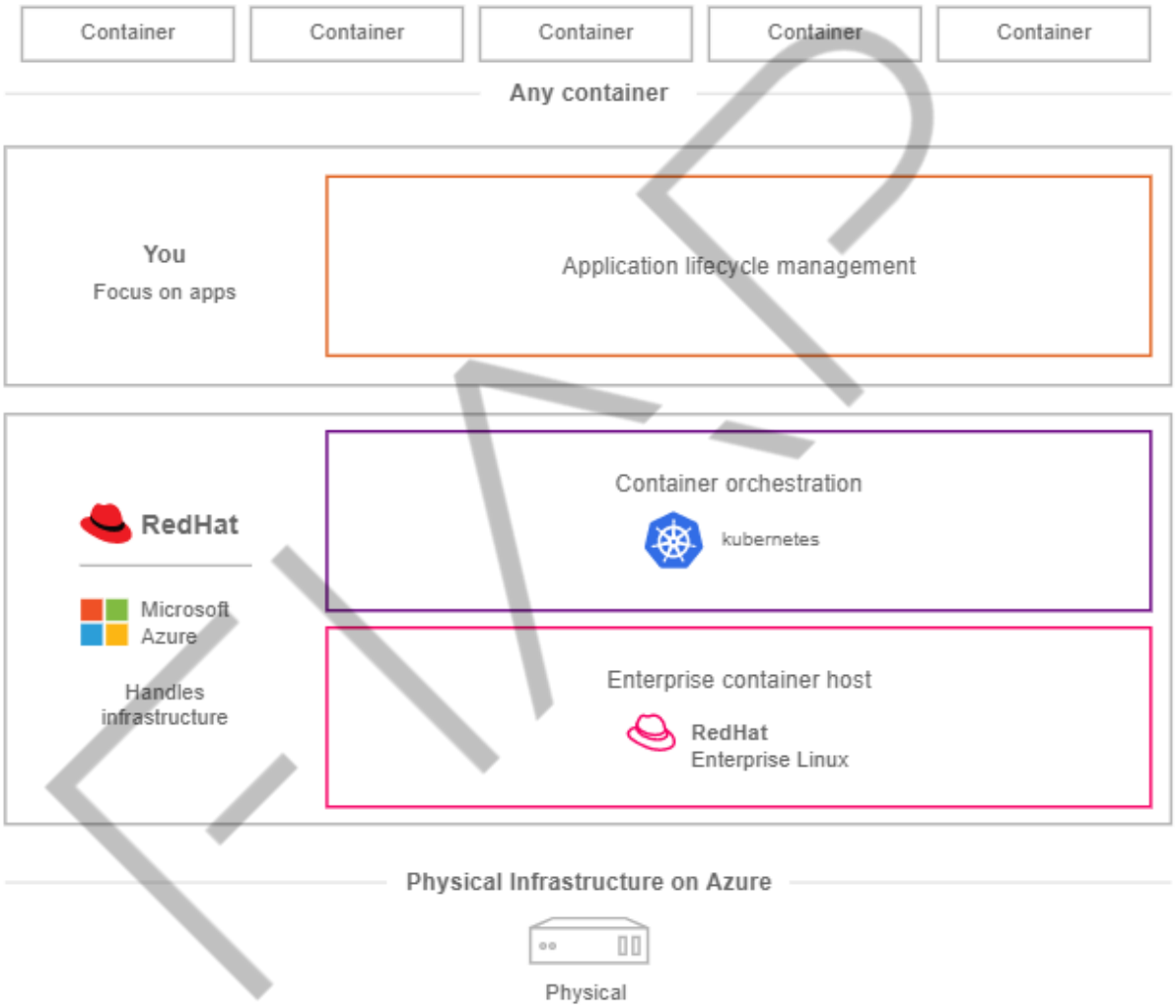

Figura 5 – Representação do Red Hat OpenShift do Azure
Fonte: OpenShift (2019)

A parceria disponibiliza o **Kubernetes de nível empresarial,** sendo desenvolvido, operado, suportado em conjunto e totalmente gerenciado na *cloud* pública Microsoft Azure. Possibilita às empresas de TI trabalhar com a Red Hat OpenShift Container Platform em seus *datacenters* e operar com a escala e a força do Microsoft Azure. Você verá, a seguir, como se apresenta a Arquitetura do OpenShift.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

2.1 Arquitetura do OpenShift

A infraestrutura do OpenShift contém um *node* para armazenar as aplicações. Como no exemplo da figura “Componentes da arquitetura do OpenShift”, temos uma estrutura baseada no Docker, em que um *node* contém aplicações que ficam dentro de seu respectivo container, agrupado pelo *pod*. Um *pod* fica hospedado em uma máquina (*node*), que faz parte do *cluster* Kubernetes, por sua vez, gerenciado pelo OpenShift.

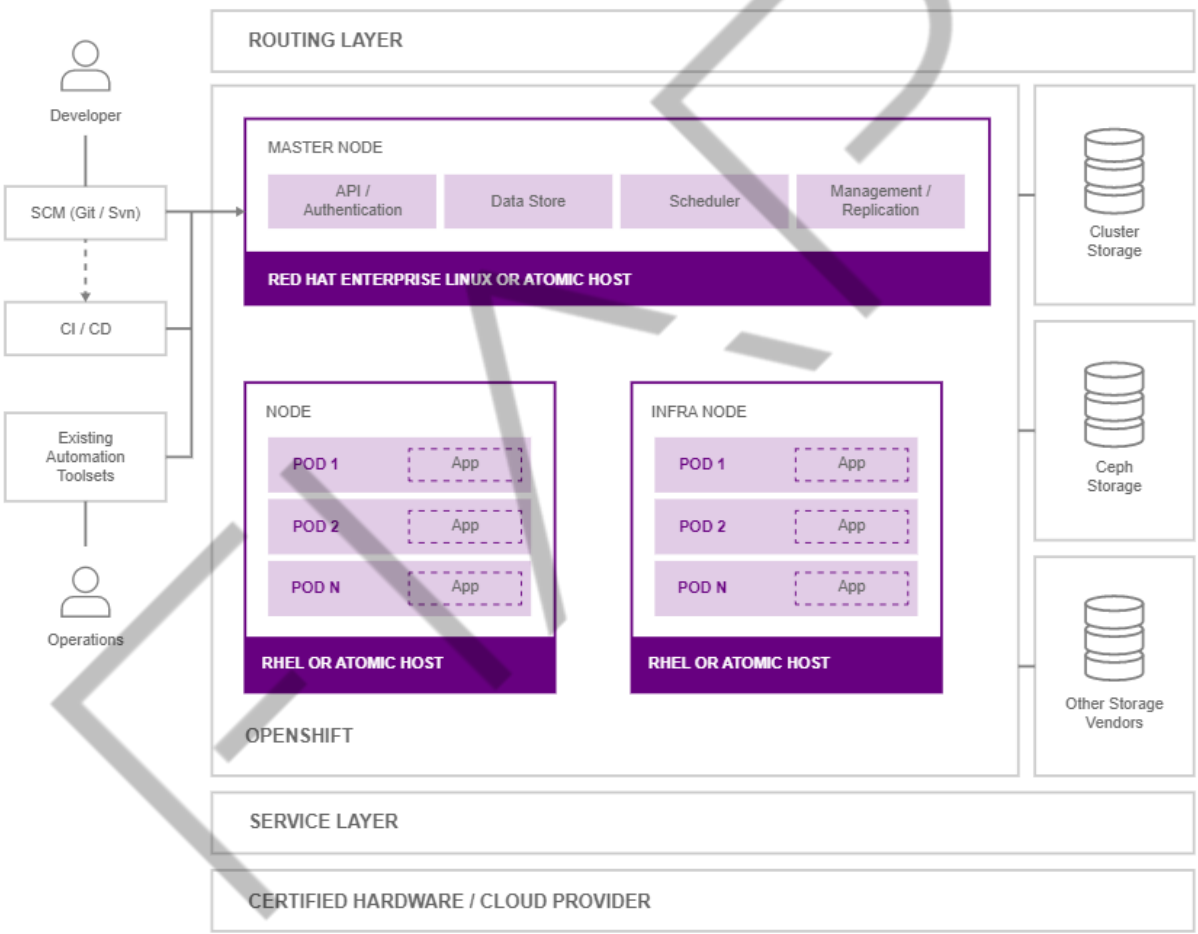

Figura 6 – Componentes da Arquitetura do OpenShift
Fonte: RedHat (2019)

Componentes do Nó Mestre (ou Node Master)

- **API/Authentication**: responsável pelo controle de acesso às APIs do OpenShift e do Kubernetes, com um processo de autenticação baseado em certificados SSL (*Secure Sockets Layer*) e no padrão OAuth.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Data Store**: armazena o estado e outras informações dos componentes do OpenShift. Geralmente, utiliza a estrutura de armazenamento etcd.
- **Scheduler**: faz a distribuição das cargas de trabalho entre os nós dos *clusters* de componentes.
- **Management/Replication**: é um processo que, além de realizar replicação, é executado, de tempos em tempos, com a finalidade de coletar informações de estado dos elementos do *cluster* e dos demais componentes, atualizar o estado e armazenar esses estados no *Data Store* (etcd). O processo Management/Replication é gerenciado por *controllers*, sendo os principais *controllers:* Replication Controller, Endpoint Controller, Namespace Controller e Service Account Controller.

Em seguida, você vai entender como funciona o OpenShift.

2.2 Funcionamento do OpenShift

Assim como o Kubernetes, o OpenShift também funciona sobre um sistema baseado em camadas no qual cada camada possui uma determinada responsabilidade. O conjunto de camadas é o que possibilita a disponibilidade das diversas funcionalidades do OpenShift.

Para rodar, o OpenShift utiliza um cluster baseado no Kubernetes em que seus componentes são organizados como microsserviços e disponibiliza dentro de sua infraestrutura um nó master, que hospeda os principais componentes do OpenShift. No diagrama “Ferramentas do Openshift”, temos uma visão geral das ferramentas que são integradas e orquestradas pelo OpenShift:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

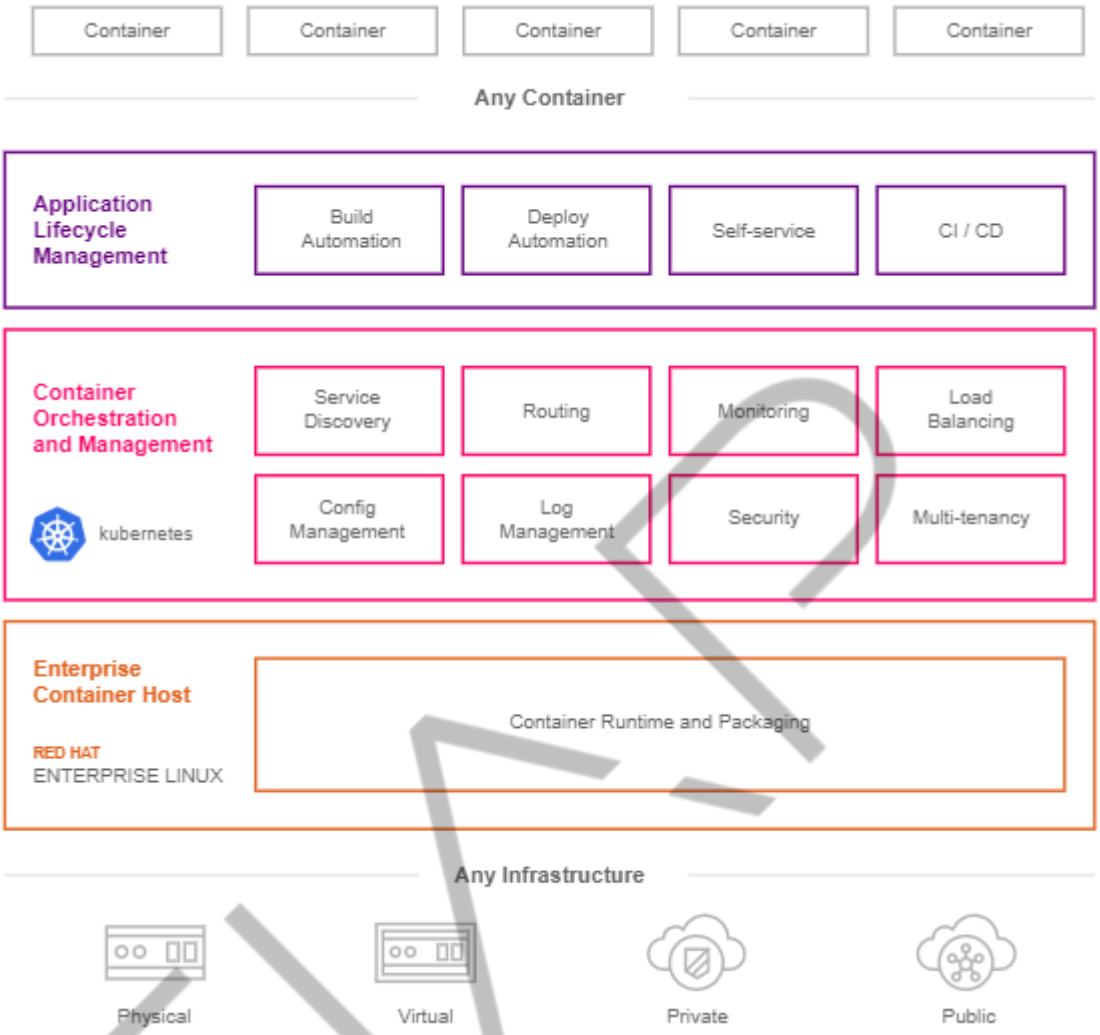

Figura 7 – Ferramentas do OpenShift
Fonte: RedHat (2019)

Como você pode observar no diagrama, o OpenShift apresenta-se como uma ferramenta qualificada para **controlar todo o ciclo de vida de uma aplicação** e para o **gerenciamento de containers**.

Em se tratando de tecnologia em nuvem, o gerenciamento não deve ocorrer apenas para containers, é preciso que haja também uma forma de gerenciar os recursos que compõem a infraestrutura. Para isso, existe o OpenStack.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

3 OPENSTACK

OpenStack é um **gerenciador de recursos de ambiente de nuvem** que mostra aos seus usuários a possibilidade de gerenciar um container da mesma forma que uma máquina virtual. Oferece **Infraestrutura como Serviço** (*IaaS – Infrastructure as a Service),* possibilizando **compartilhamento de hardware** no ambiente, **escalabilidade**, **isolamento** e **segurança**. O OpenStack é mantido por um conjunto de empresas e desenvolvedores dos quais fazem parte empresas como Red Hat, Canonical e Cisco. “O OpenStack é uma plataforma de orquestração de recursos em nuvem computacional que, desde 2014, tem incorporado módulos e ferramentas para uso de containers” (OPENSTACK, 20120).

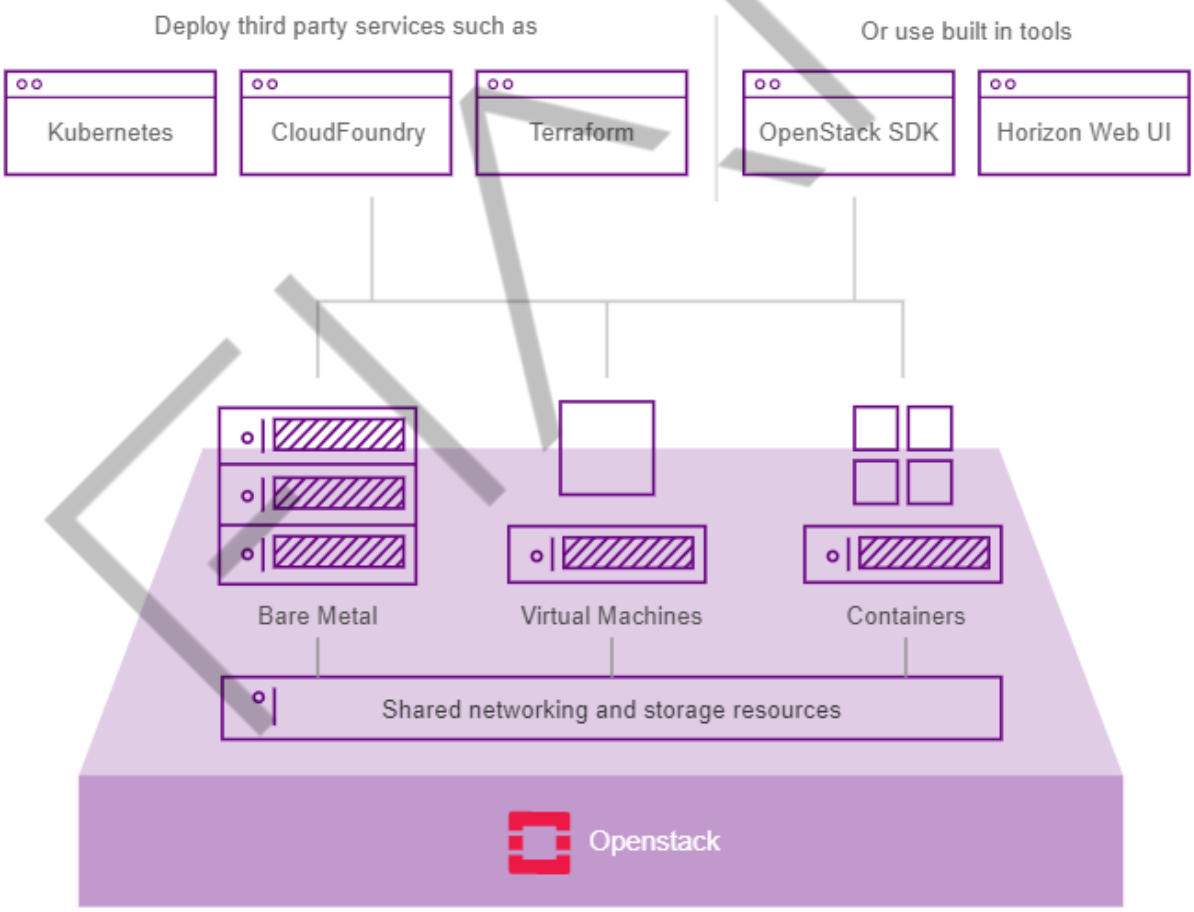

Figura 8 – Representação do OpenStack
Fonte: OpenStack (2019)

Suporta o provisionamento em três tipos *hosts* diferentes:

- **Bare-metal (servidor físico):** sem virtualização, o hardware é totalmente dedicado, entregue sob demanda.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Máquina virtual**: virtualização tradicional em que a máquina aparece como um computador independente, inicializa kernel do SO padrão e executa um aplicativo.
- **Containers**: abordam isolamento de recursos, nos quais os aplicativos compartilham um kernel comum.

Disponibiliza para os administradores um painel com interface web que permite controle total e capacitação de seus usuários com o objetivo de atingir melhor provisionamento de recursos. A arquitetura do OpenStack é composta por módulos que possibilitam a implantação por conteinerização.

3.1 Arquitetura do OpenStack

A plataforma do OpenStack foi projetada para ser escalável e flexível e seu *core* (núcleo) é formado por componentes e serviços. Fornece máquinas virtuais inicializáveis, rede, armazenamento em bloco, armazenamento de objetos e assim por diante.

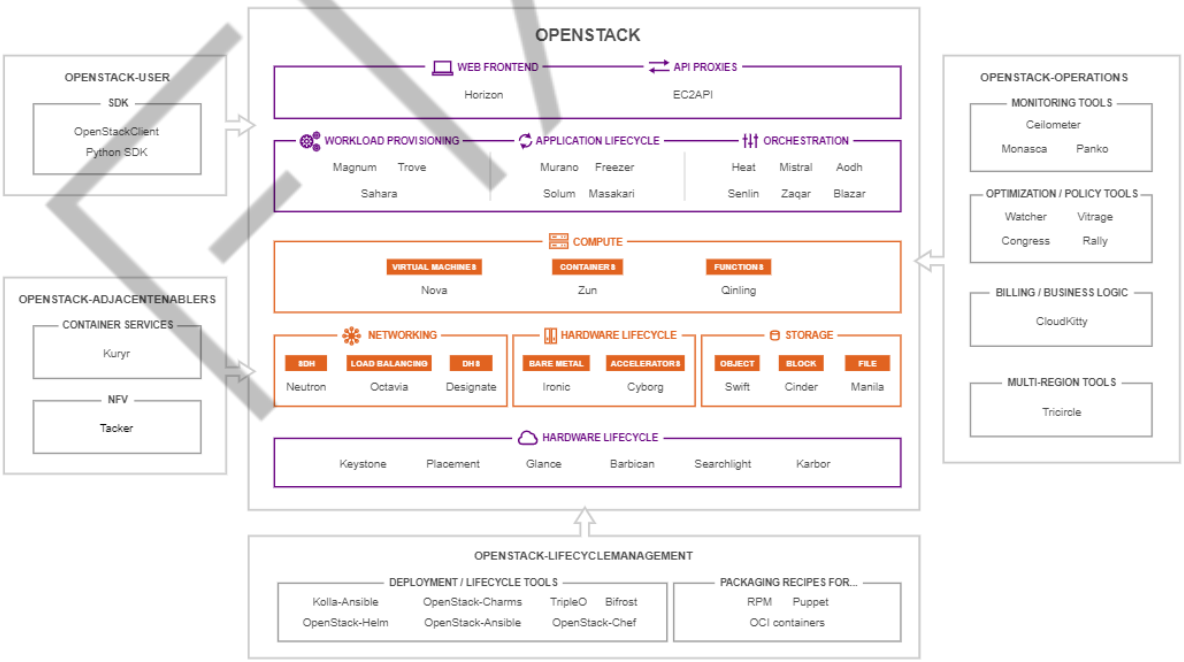

Figura 9 – OpenStack Landscape
Fonte: OpenStack (2019)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Componentes**: responsáveis pelo gerenciamento de falhas e de serviços, orquestração e por garantir alta disponibilidade dos aplicativos do usuário. Implementam as funcionalidades base da infraestrutura de nuvem.
- **Serviços**: responsáveis pela integração dos diversos componentes para fornecer uma plataforma de IaaS completa.

Módulo	Nome	Função
Compute	NOVA ZUN QINLING	Compute Service Containers Service Functions Service
Networking	NEUTRON OCTAVIA DESIGNATE	Networking Load balancer DNS service
Storage	SWIFT CINDER MANILA	Object store Block Storage Shared filesystems
Hardware Lifecycle	IRONIC CYBORG	Bare Metal Provisioning Service Lifecycle management of accelerators

Quadro 1 – Componentes e serviços do OpenStack
Fonte: OpenStack (2019)

Na página oficial do OpenStack é possível encontrar os demais componentes e serviços, bem como suas funcionalidades e indicação de uso. Agora, vamos ao funcionamento do OpenStack!

3.2 Funcionamento do OpenStack

O OpenStack, como uma infraestrutura de nuvem, é sustentado pelos pilares: processamento (***compute***), rede (***networking***) e armazenamento (***storage***). Esse SO faz uso de APIs com mecanismos comuns de autenticação para realizar tarefas de **gerenciamento** e **provisionamento**.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

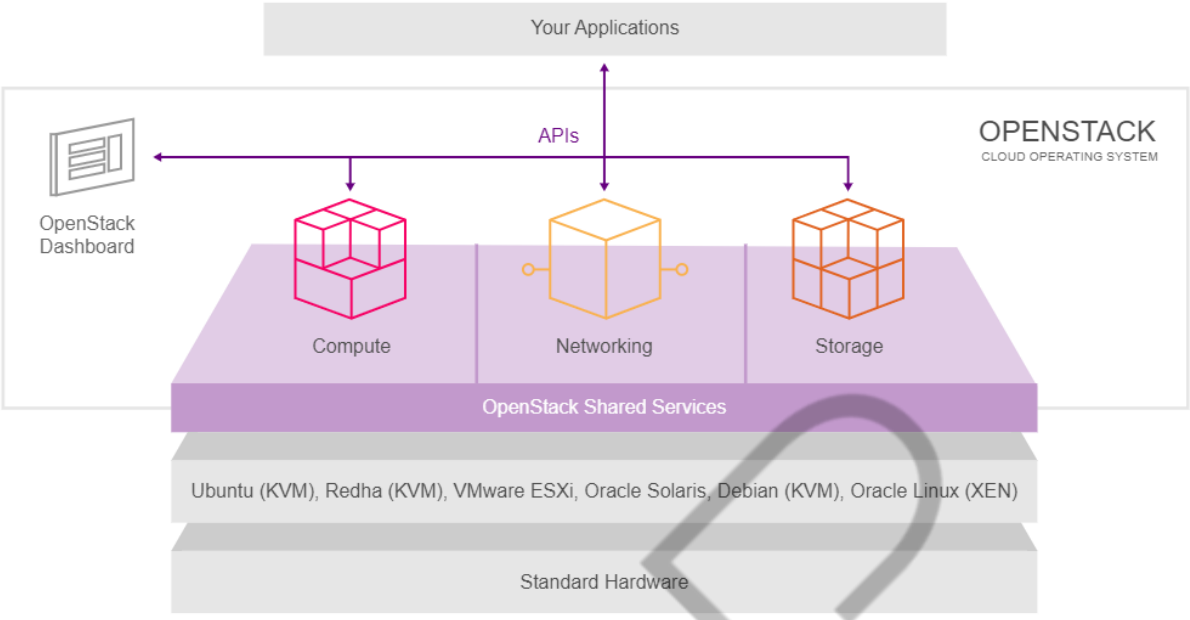

Figura 10 – Pilares da infraestrutura OpenStack
Fonte: OpenStack (2019)

Os serviços permitem a **personalização** de acordo com as necessidades como, por exemplo, conectar e reproduzir componentes específicos. Configurando adequadamente o serviço *bare metal* com os serviços *Compute* e *Networking,* é possível provisionar máquinas virtuais e físicas por meio da API do serviço *Compute*.

3.3 Instalando o DevStack

O DevStack é uma implementação simplificada do OpenStack quem implementa os módulos básicos de funcionamento como gestam de rede, armazenamento e máquinas além de disponibilizar o módulo Horizon para permitir a interação por meio de uma interface visual.

A primeira coisa que você irá precisar para testar o OpenStack é de uma máquina virtual, para isso nós vamos baixar e instalar o virtualbox, um software gratuito de virtualização mantido pela Oracle.

Link para download https://www.virtualbox.org/wiki/Downloads

Você também irá precisar de uma imagem do Ubuntu 20.04 que será instalado na máquina virtual e servirá de base para o DevStack ser instalado.

Link para donwload: http://linorg.usp.br/ubuntu-releases/20.04/

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

De preferência, opte pelas imagens para servidor, elas não possuem interface gráfica, porém são muito mais leves para serem executadas.

Após o processo de instalação, iremos criar a nossa primeira máquina virtual, tenha em mente que essa máquina precisará ter no mínimo 4 GB de memória RAM, 10 GB de espaço em disco e 2 núcleos de processamento para podermos rodar o DevStack.

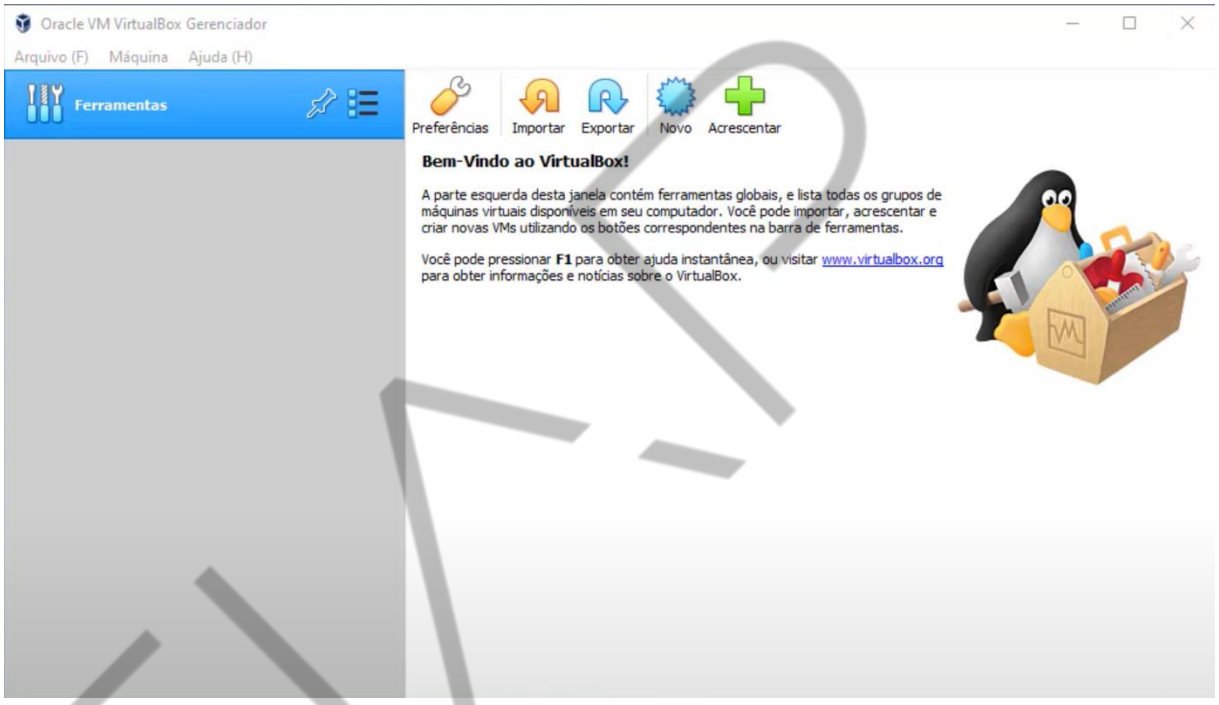

Figura 11 – Tela inicial do VirtualBox
Fonte: Elaborado pelo autor (2022)

Na tela inicial do virtualbox, selecione o ícone novo para que possamos criar uma nova máquina virtual.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

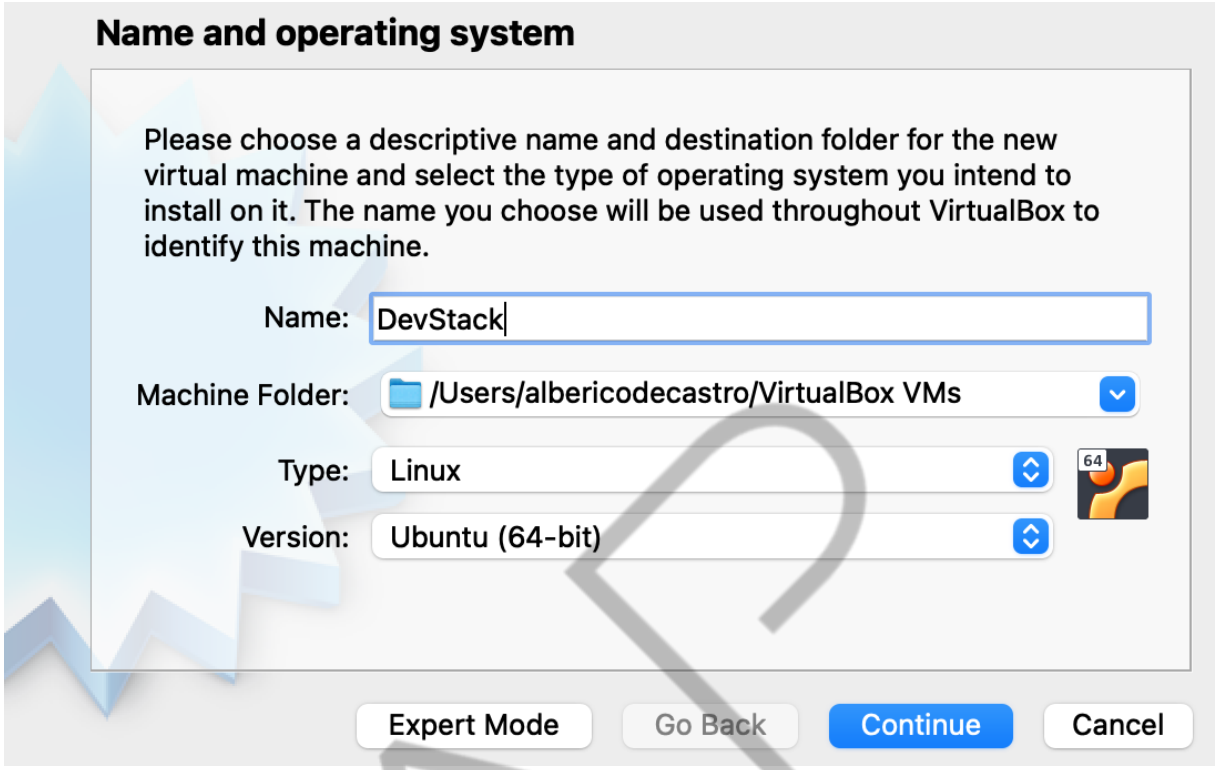

Figura 12 – Criando uma VM nova no VirtualBox
Fonte: Elaborado pelo autor (2022)

Entre com o nome para VM, selecione o tipo Linux, especifique a versão do sistema operacional, como o Ubuntu 64 bits. Clique na opção **continuar** e, em seguida, preencha a quantidade de memória que irá disponibilizar. Nesse caso, você pode preencher com 4096 MB que seriam equivalentes aos 4GB de memória RAM necessários.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

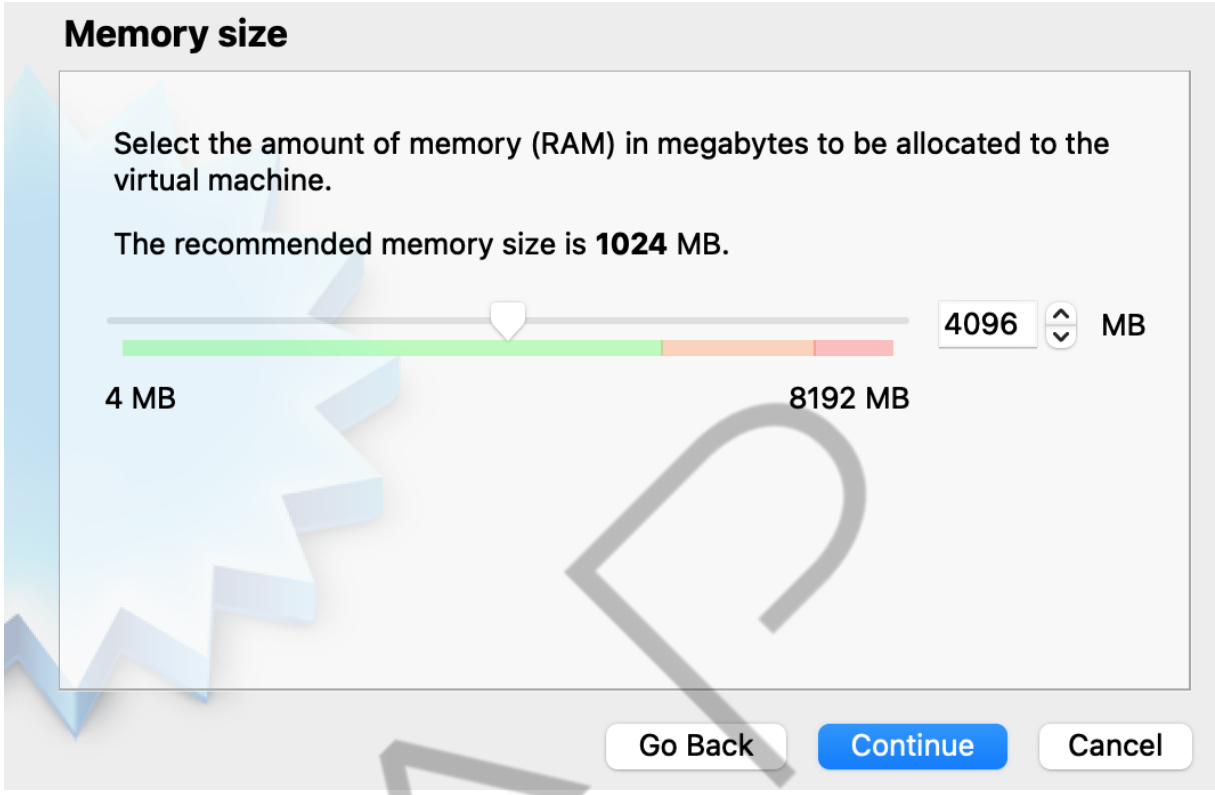

Figura 13 – Configurando a quantidade de ram da VM no VirtualBox
Fonte: Elaborado pelo autor (2022)

A próxima etapa será especificação da quantidade de disco que será disponibilizado para a máquina, selecione a opção continuar até que você chegue na tela que permite configurar a quantidade GB que serão disponibilizados para a máquina virtual. Lembre-se de configurar no mínimo 10 GB de espaço para a criação da VM.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

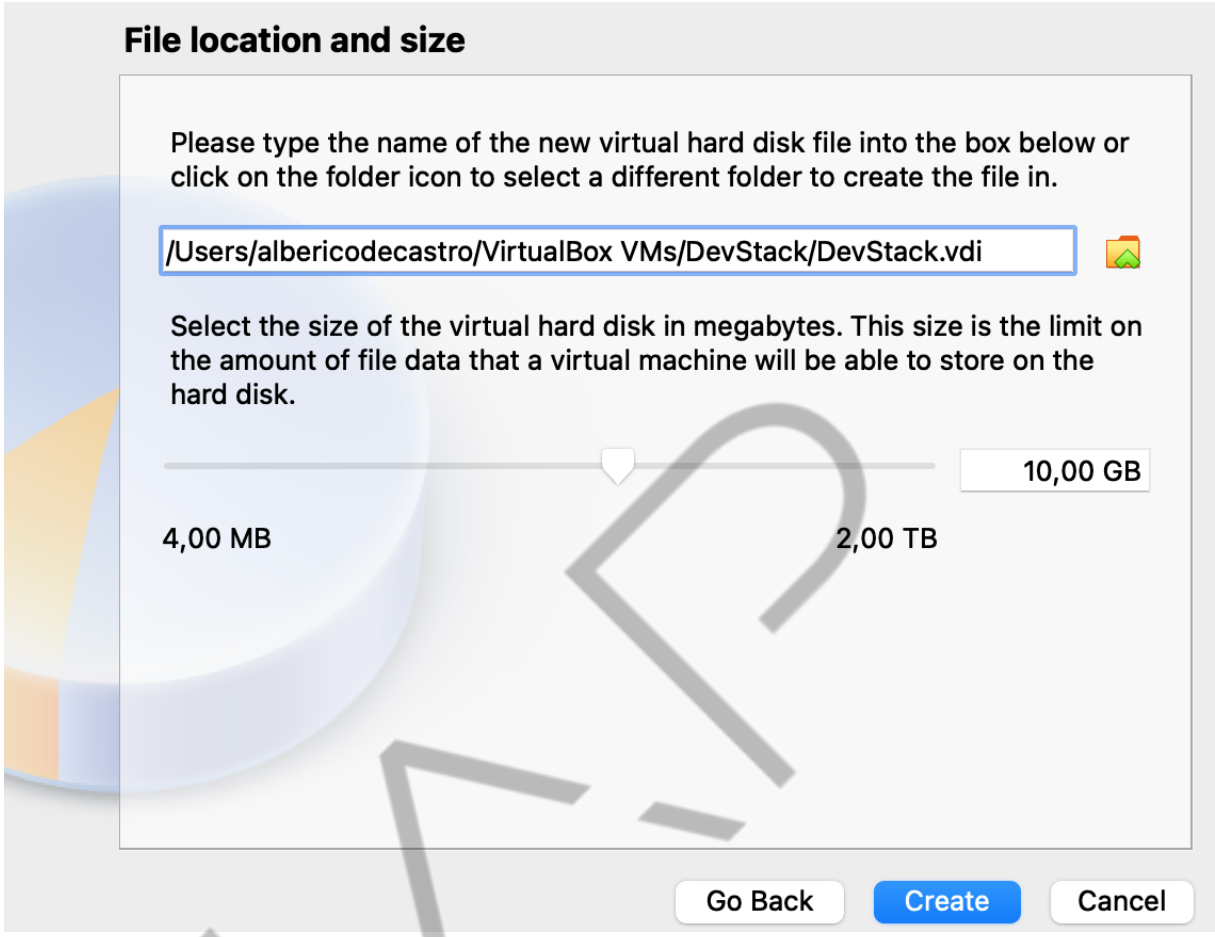

Figura 14 – Configurando a quantidade de espaço em disco da VM no VirtualBox
Fonte: Elaborado pelo autor (2022)

Agora que eu vi está criada nós estamos a apenas um passo de começarmos o processo de instalação do DevStack. Antes de ligar a máquina virtual precisamos configurar a placa de rede para utilizar um compartilhamento da interface de física de rede. Para isso, clique no ícone de configuração para abrir as opções de configuração da vm e navegue para as configurações de rede, e depois altere a interface de rede de NAT para Bridged Adapter.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

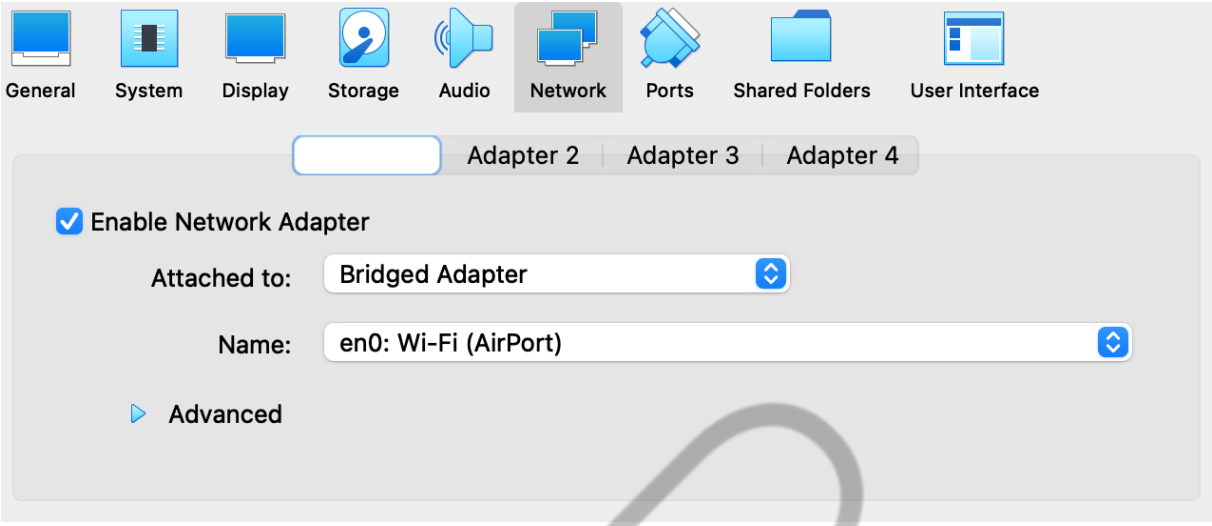

Figura 15 – Configurando a interface de rede da VMno VirtualBox
Fonte: Elaborado pelo autor (2022)

Com essas configurações feitas, você pode prosseguir com a instalação do Ubuntu; durante o processo basta selecionar o seu idioma e utilizar as configurações padrões para todo o resto do processo de instalação.

Quando a sua VM terminar de instalar o sistema operacional, execute o comando “ip addr” para checar qual é o endereço de IP da sua máquina virtual. Guarde o IP, pois iremos utilizá-lo para algumas configurações do nosso sistema.

O nosso primeiro passo será executar a atualização do sistema operacional por meio do comando:

```
sudo apt-get update && sudo apt-get upgrade -y
```

Em seguida, você deverá criar um usuário para a aplicação do DevStack e garantir que ele execute comandos sem que seja preciso informar a senha. Esse passo é necessário, pois toda a instalação será realizada por meio de script.

```
sudo useradd -s /bin/bash -d /opt/stack -m stack
echo "stack ALL=(ALL) NOPASSWD: ALL" | sudo tee /etc/sudoers.d/stack
```

Agora você precisa trocar o seu usuário de acesso para usuários que acabamos de criar. Para isso, basta rodar o comando a seguir:

```
sudo su - stack
```

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Chegou a hora de baixar a aplicação do DevStack. Após o download, iremos navegar para dentro da pasta que foi baixada e criar um arquivo chamado: local.conf, que irá armazenar as configurações de senha para o sistema que estamos instalando.

```
git clone https://opendev.org/openstack/devstack
cd devstack
vim local.conf
```

Após a execução do último comando, você será levado para um editor de texto, onde deverá digitar o conteúdo apresentado a seguir, alterando apenas a senha do ADMIN_PASSWORD para a senha que você desejar.

Para digitar os dados basta digitar a tecla: **i** para entrar em modo de inserção de dados, e para sair do editor basta digitar **ESC** e depois **wq**, seguido de **Enter**.

```
[[local|localr]]

ADMIN_PASSWORD=AltereEssaSenha
DATABASE_PASSWORD=$ADMIN_PASSWOCinder
RABBIT_PASSWORD=$ADMIN_PASSWORD
SERVICE_PASSWORD=$ADMIN_PASSWORD
```

Último passo para o processo de instalação é executar o instalador em si. Para isso, basta executar o comando:

```
./stack.sh
```

O processo leva alguns minutos para ser concluído. Quando o processo estiver finalizado, basta acessar no seu navegador web, o site: https://ip_da_vm/dashboard e você será levado para a página de login do OpenStack.

Vale lembrar que o IP que deve ser preenchido, no link de acesso, é o mesmo IP que nós checamos quando inicializamos a máquina virtual. Outro detalhe útil é que o usuário da aplicação é **admim** e a senha é a mesma que você informou no arquivo de texto que editamos.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Figura 16 – Tela de boas-vindas do OpenStack
Fonte: Elaborado pelo autor (2022)

Sinta-se à vontade para brincar e experimentar um ambiente de cloud privada. Você irá notar que é uma experiência de uso bem parecida com uma cloud pública.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

4 ENTENDENDO ALGUMAS COMPARAÇÕES

4.1 Kubernetes e Docker

Muitos confundem os papéis e responsabilidades do Kubernetes e do Docker e tentam fazer comparações, mas o correto é comparar Kubernetes com “Docker Swarm”, que são ferramentas de orquestração de *cluster*.

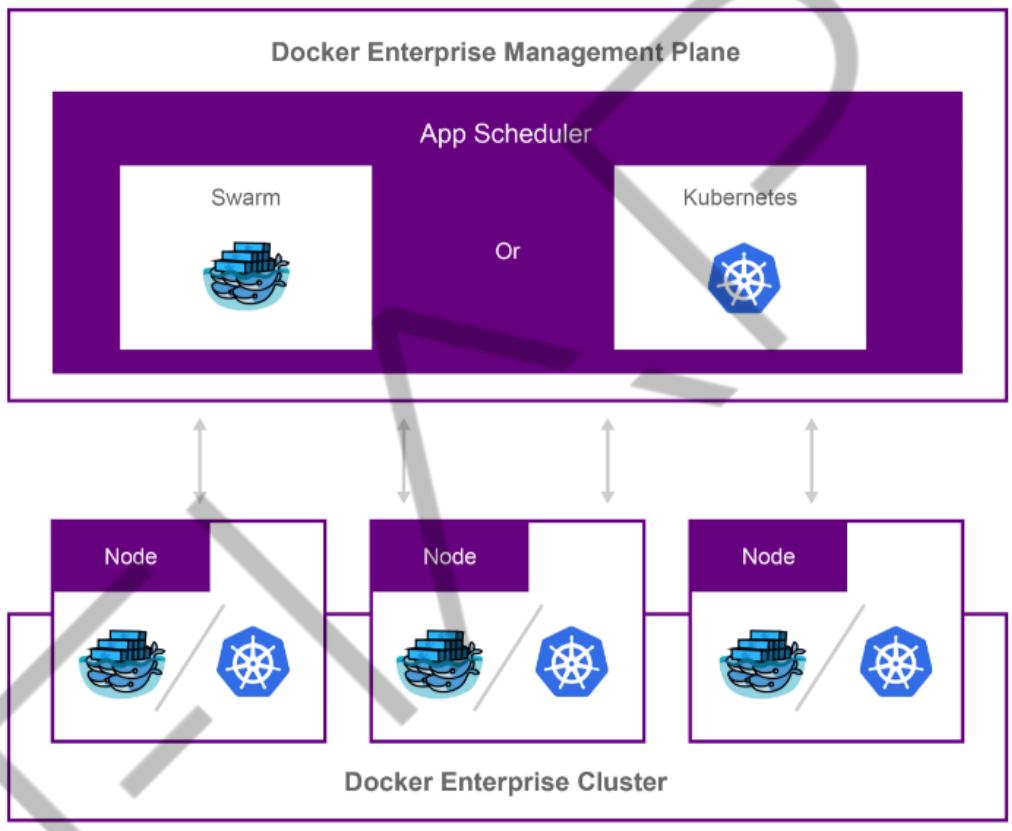

Figura 17 – Papel do Kubernetes e do Docker Swarm
Fonte: Docker (2019)

O papel do Kubernetes é utilizar o Docker para criar containers nos nós do *cluster* e tem como responsabilidades controlar, gerenciar e monitorar o estado dos containers Docker ao longo do *cluster*.

O Docker Swarm é nativo do Docker e desempenha as mesmas funcionalidades do Kubernetes, mas apresenta algumas variações e diferenças técnicas e funcionais. Os pontos de atenção ao utilizar o Docker Swarm são:

- Funcionalidade limitada.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- Tolerância à falhas limitada.
- Os serviços podem ser dimensionados manualmente.

Devido a essas e outras características, o Kubernetes é considerado mais confiável no mercado de tecnologia. O Kubernetes conta com recursos de dimensionamento automático e políticas de alta disponibilidade. Por esses motivos, é muito usado por organizações de grande porte que possuem aplicativos complexos que necessitam utilizar centenas de milhares de containers no ambiente de produção.

4.2 Kubernetes e Microsserviços

Diante do que você já viu até aqui, é possível observar que a maioria dos recursos oferecidos pelo **Kubernetes** também é essencial para o contexto dos microsserviços. Por isso, o uso do Kubernetes **gerenciando um container Docker** tornou-se uma prática extremamente poderosa para a implantação de microsserviços, especialmente para os microsserviços de grande porte.

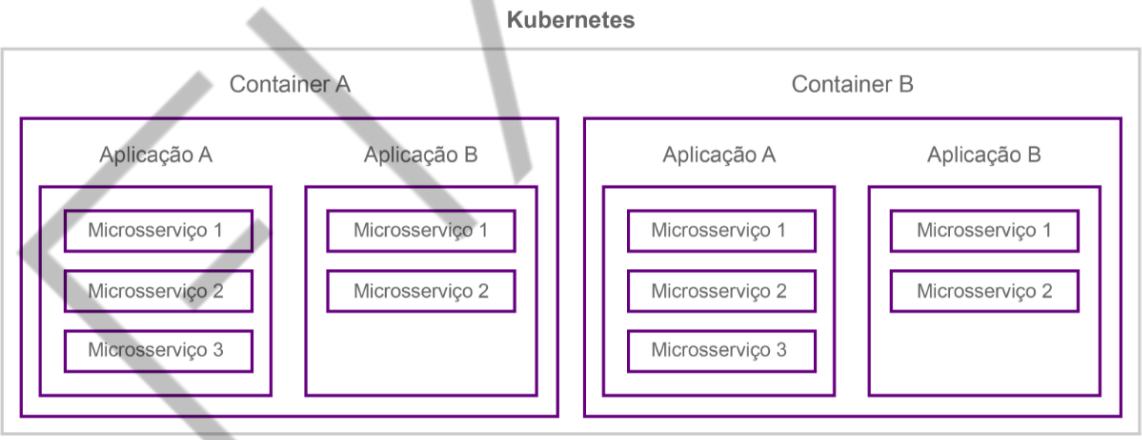

Figura 18 – Relação do Kubernetes e Microsserviços
Fonte: Elaborado pela autora, adaptado por FIAP (2019)

4.3 Kubernetes x OpenShift x OpenStack

A realidade é que Kubernetes, OpenShift e OpenStack não podem ser comparados por serem tecnologias independentes e diferentes em sua essência, usados separadamente uns dos outros. Então, veja um resumo das características de cada uma das tecnologias em questão:

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

- **Kubernetes:** desempenha o papel de sistema operacional no nível de containers com a responsabilidade de executar tarefas de orquestração de containers.
- **OpenShift:** incorporou o Kubernetes para oferecer funcionalidades além de gerenciamento de containers e foi otimizado para rodar em infraestrutura de *cloud* pública ou privada.
- **OpenStack:** responsável por gerenciar grandes quantidades de recursos computacionais que envolvem processamento, armazenamento e rede. Em suma, oferece uma plataforma para gerenciar a infraestrutura que é responsável por executar seus servidores.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

5 HANDS-ON – IMPLEMENTAÇÃO E CONFIGURAÇÃO DO KUBERNETES USANDO O PORTAL AZURE

O objetivo deste hands-on é implementar e configurar o serviço de Kubernetes e implantar uma aplicação em contêiner.

Para implementar essa função, será necessário:

- Uma conta na nuvem Azure.

A seguir a sequência de ações que deve ser executada para disponibilizar uma função serverless:

Na plataforma Azure, escolher o serviço “Kubernetes services” (Figura 19).

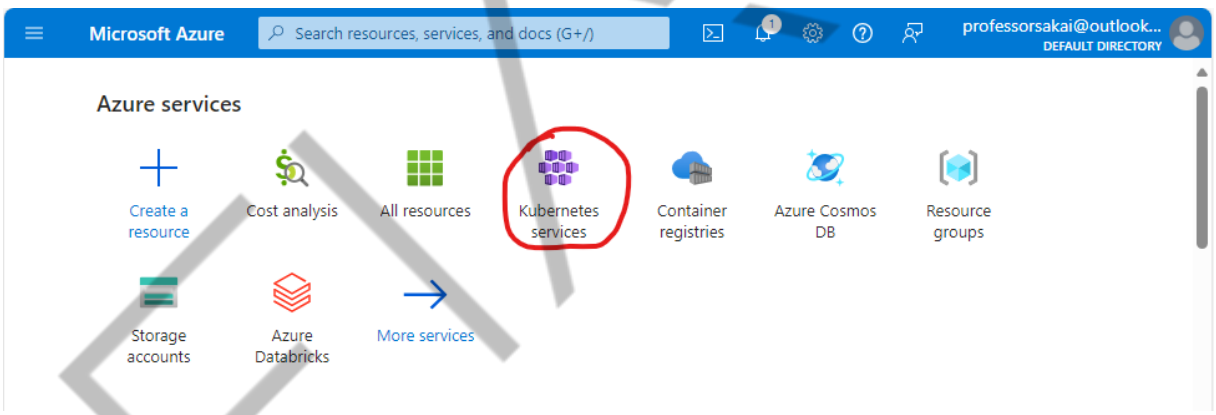

Figura 19 - Kubernetes Service
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Na próxima tela, clicar em “+ Create” ou “Create”, qualquer opção levará a mesma configuração. Escolher a opção “Create Kubernetes Cluster” para iniciar a criação de um cluster de máquinas gerenciadas pelo serviço Kubernetes (Figura 20).

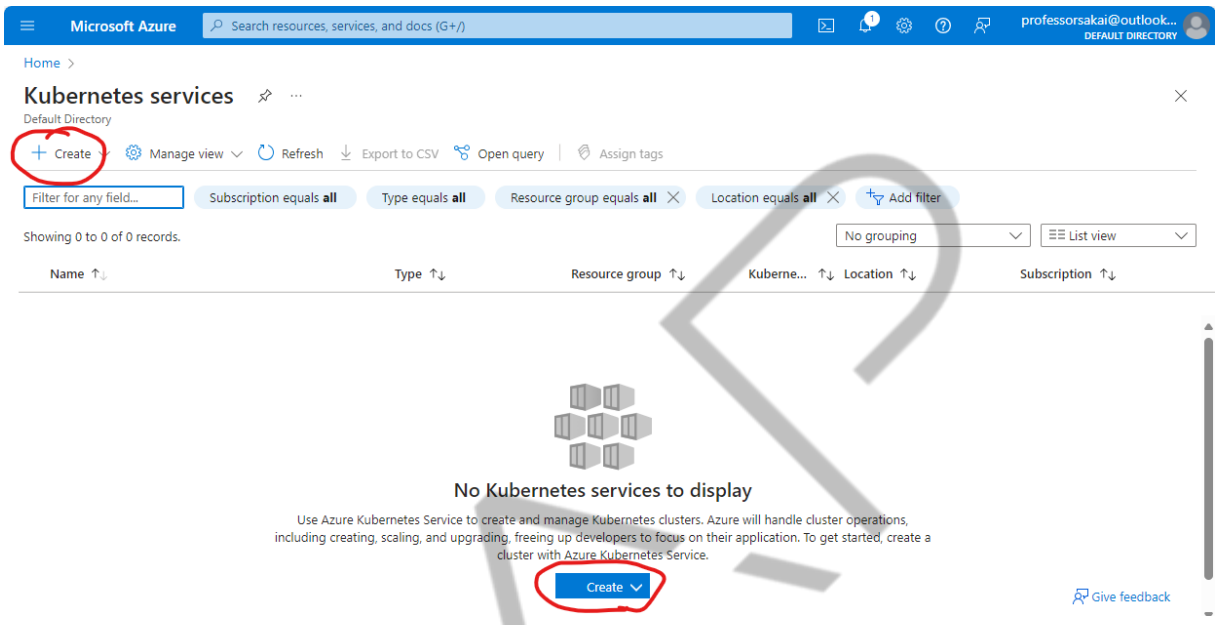

Figura 20 - Create Kubernetes Service Cluster
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

A partir desse ponto, os detalhes de configuração do cluster kubernetes devem ser preenchidos. As etapas a seguir devem ser seguidas pontualmente.

Configurações básicas, como conta, resource group, nome do cluster, são preenchidas nessa etapa (Figura 21).

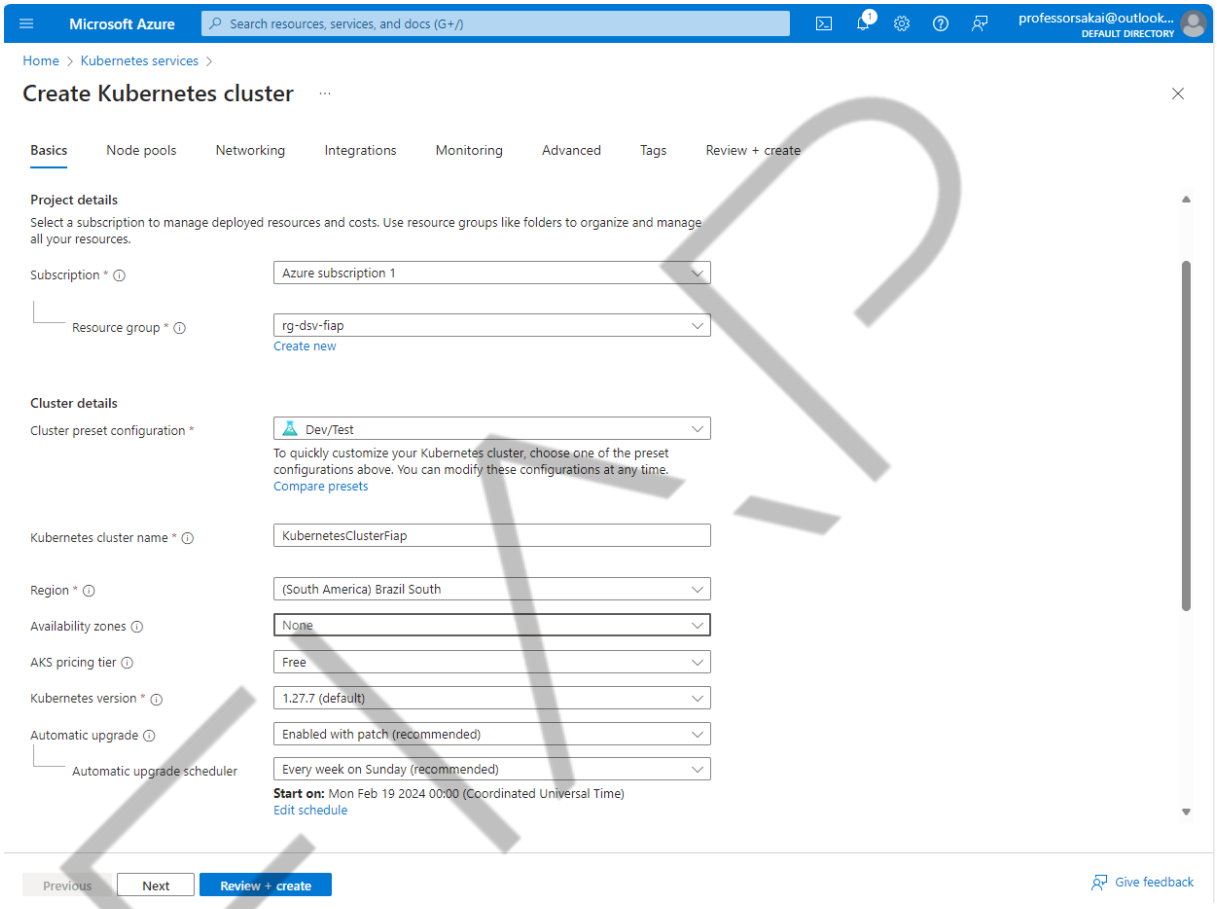

Figura 21 - Configurações Básicas
Fonte: Elaborado pelo autor (2024)

Subscription: Nome da conta Azure criada pelo aluno.

Resource Group: Pode ser um RG já existente ou criar um novo. Os resource groups (RGs) são estruturas lógicas para agrupar recursos em um mesmo local.

Cluster preset configuration: Escolha a configuração DEV/TEST para poupar recursos financeiros.

Kubernetes Cluster Name: Um nome para o cluster. “KubernetesClusterFIAP”.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Region: East US 2 (pode escolher qualquer outra região que esteja disponível). A escolha das regiões não é aleatória, deve ser levado em conta itens como custo, recursos disponíveis, localização geográfica, etc.

Availability zones: desmarcar todas as zonas de disponibilidade para não gerar custos financeiros.

Kubernetes Version: escolher a versão que está marcada como “default” que é a mais estável, nesse momento a versão 1.27.7

As outras opções deixar com as configurações padrão, conforme figura.

Na aba “Node Pools” configurar o tamanho das máquinas virtuais, selecionar a menor máquina possível para reduzir os custos financeiros. Nessa configuração foi escolhida a máquina B2s, conforme Figura 22.

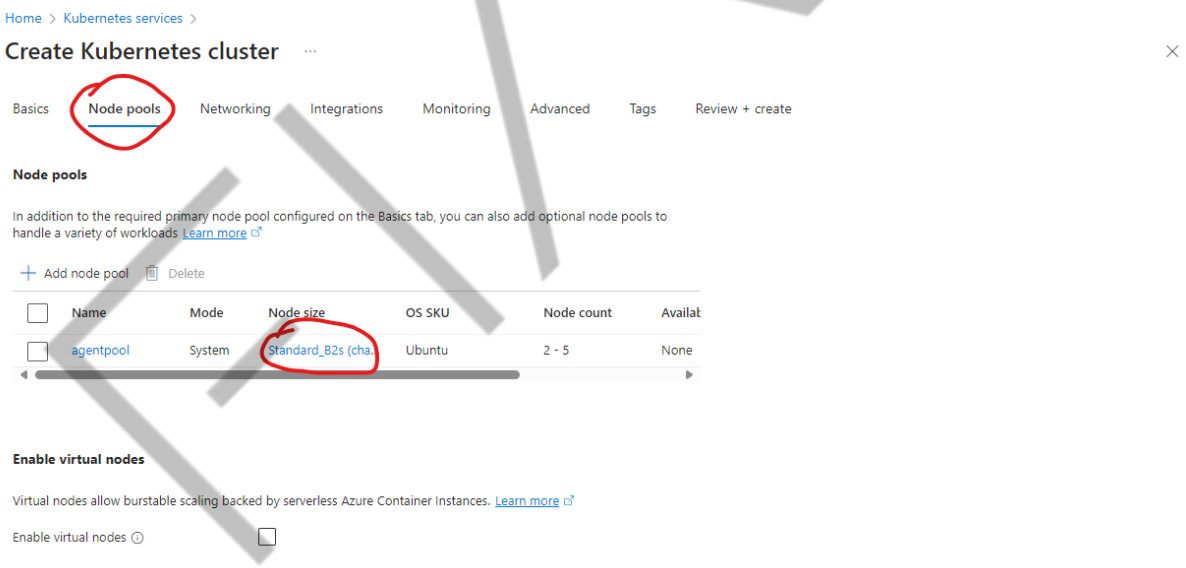

Figura 22 - Configuração dos nodes
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Manter as outras configurações sem alteração. Nesse momento, não será necessário configurá-las. Clicar em “Review + create” (Figura 23), caso todas as configurações obrigatórias estejam corretas, o botão de “Create” se tornará disponível (Figura 24). Caso contrário, será mostrado o problema a ser corrigido.

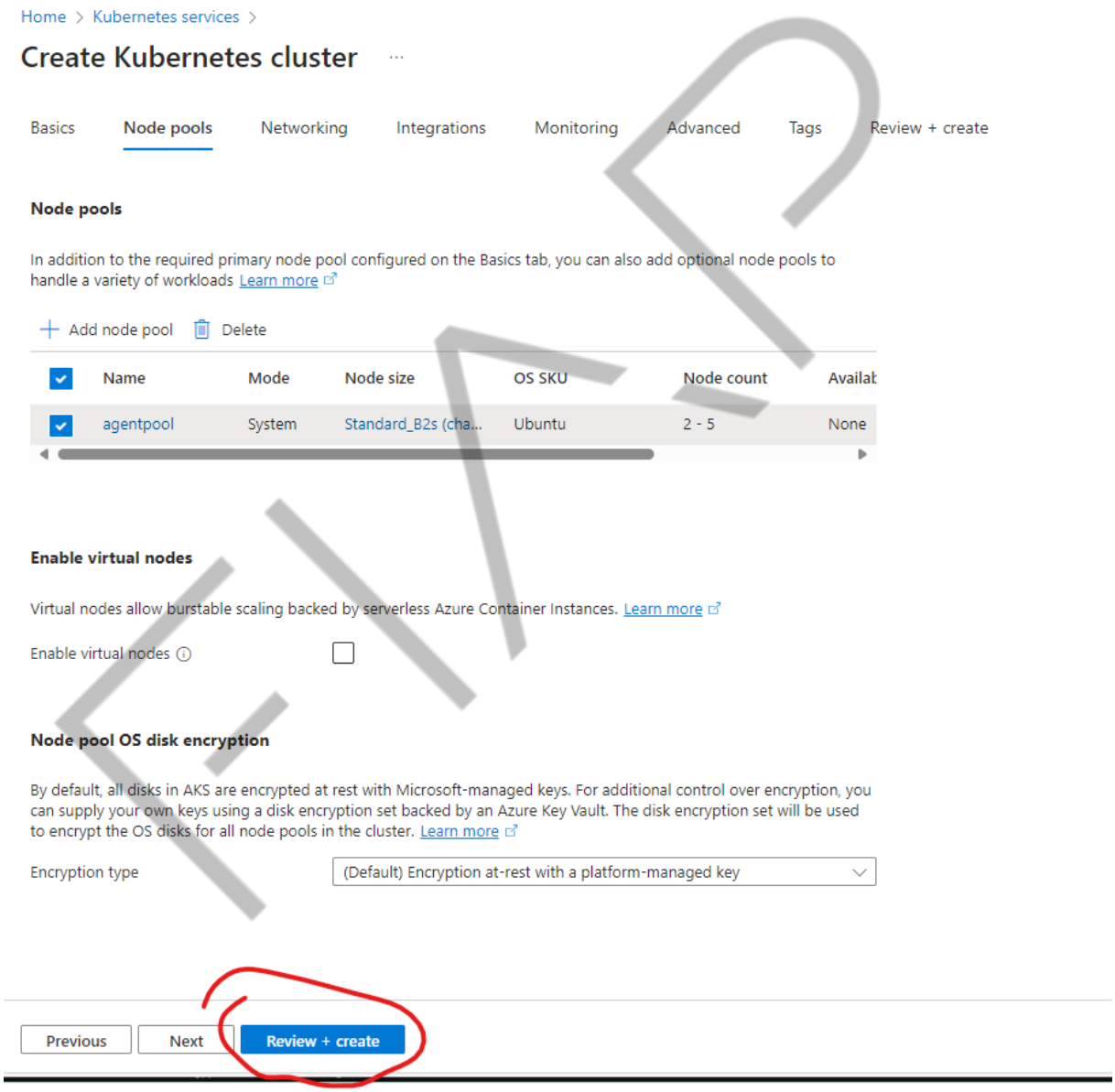

Figura 23 - Revisar e Criar
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

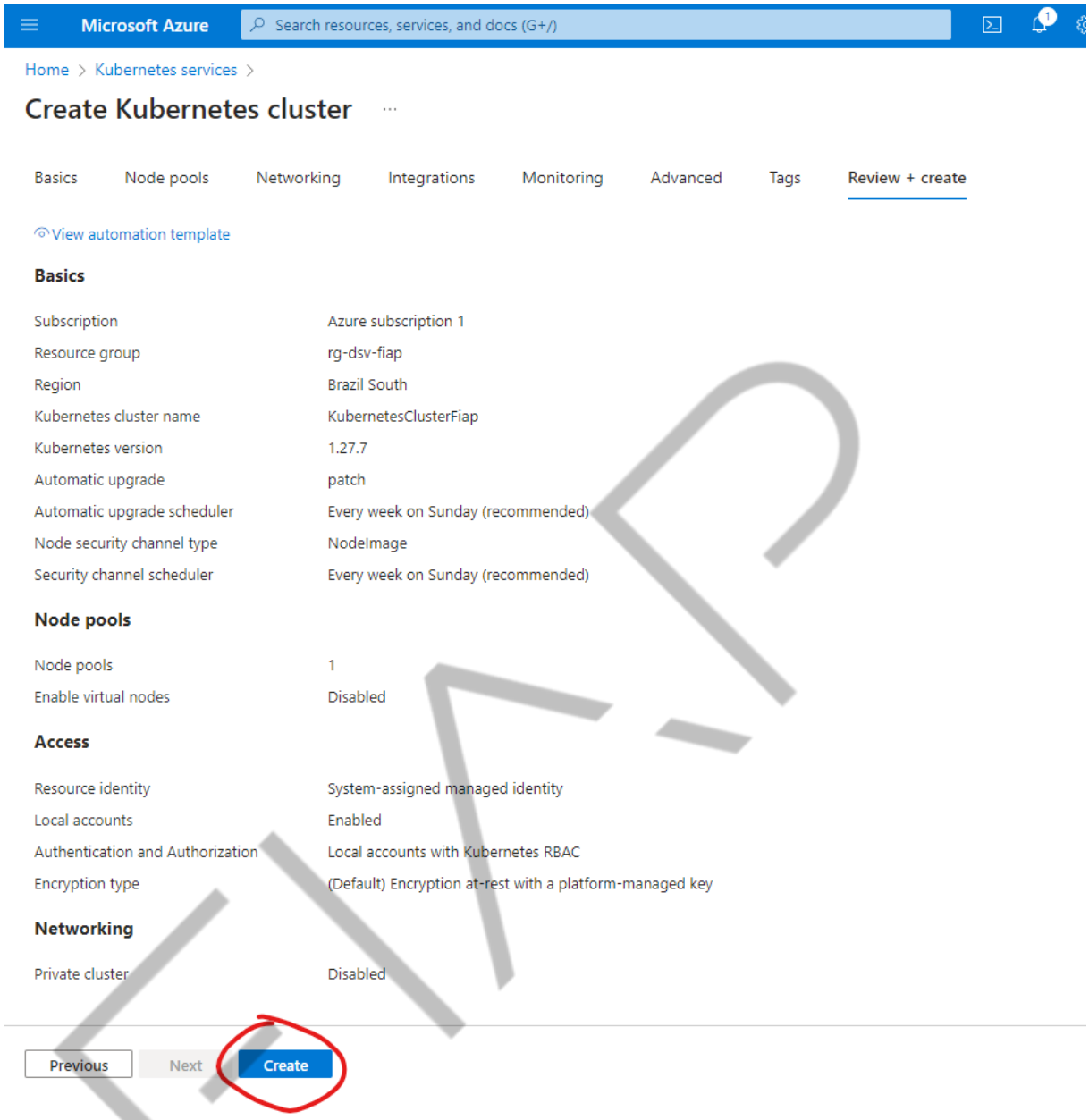

Figura 24 – Criar
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Uma tela com o “Deployment is in progress” vai aparecer conforme Figura 25. Assim que o recurso estiver disponível, será apresentada uma tela conforme Figura 26. Clicar em “Go to resource” para continuar a configuração do cluster Kubernetes”.

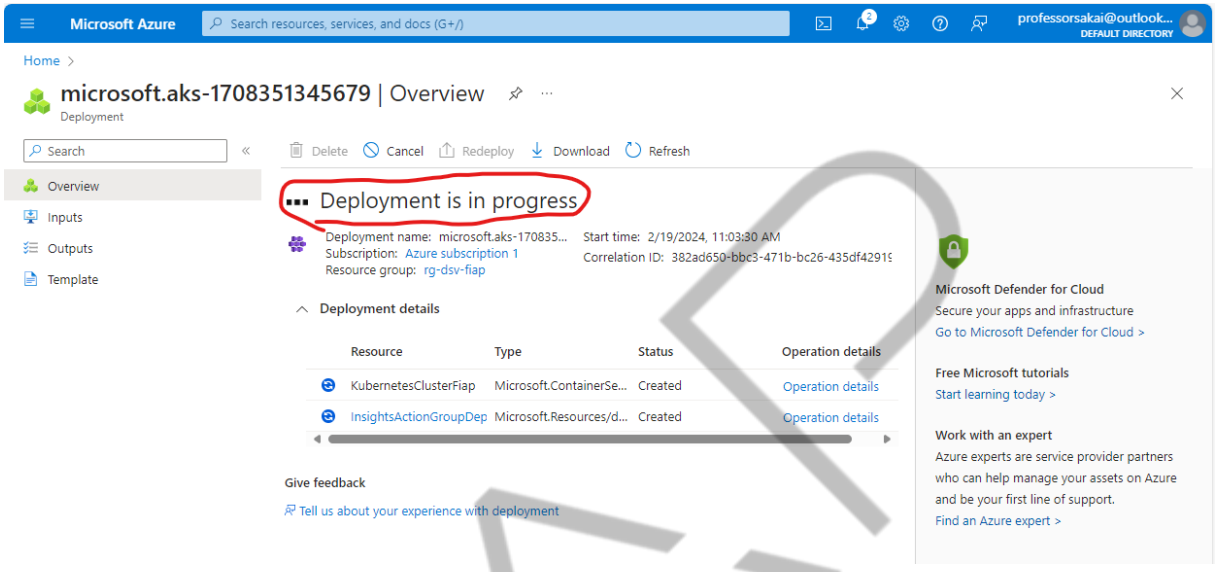

Figura 25 - Implantação do Cluster
Fonte: Elaborado pelo autor (2024)

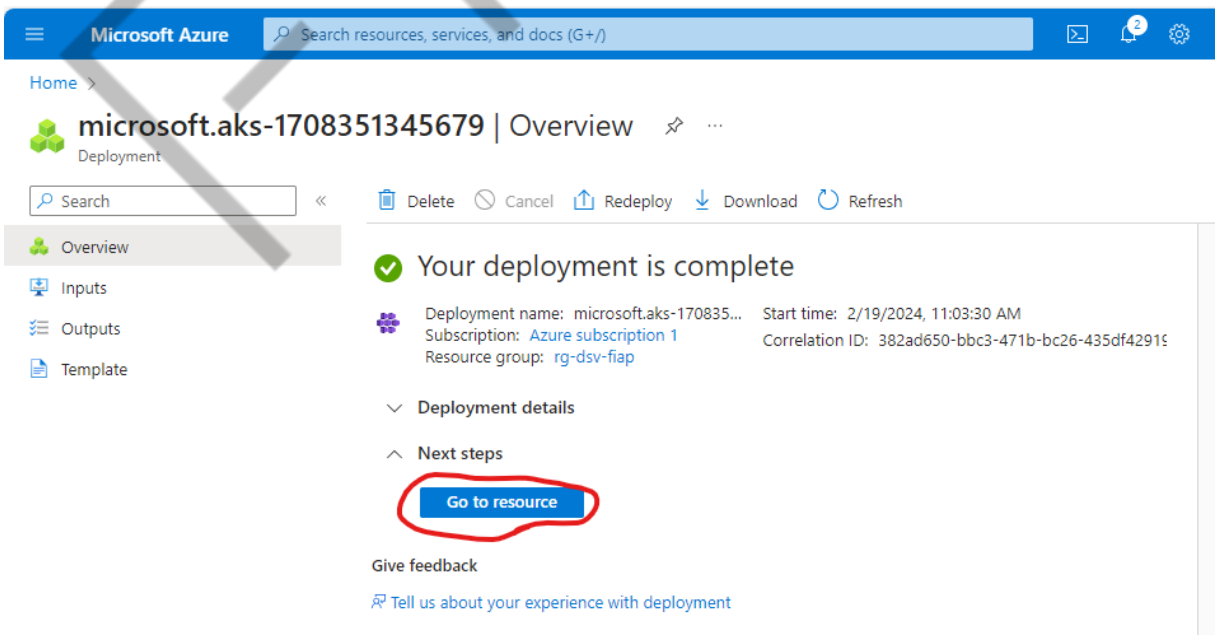

Figura 26 - Go to Resource
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Na área principal do cluster Kubernetes criado, clicar no botão no meio da página “Create a quick start application”, conforme Figura 27.

Com essa opção será criada uma aplicação simples em um container que será disponibilizado no Cluster Kubernetes.

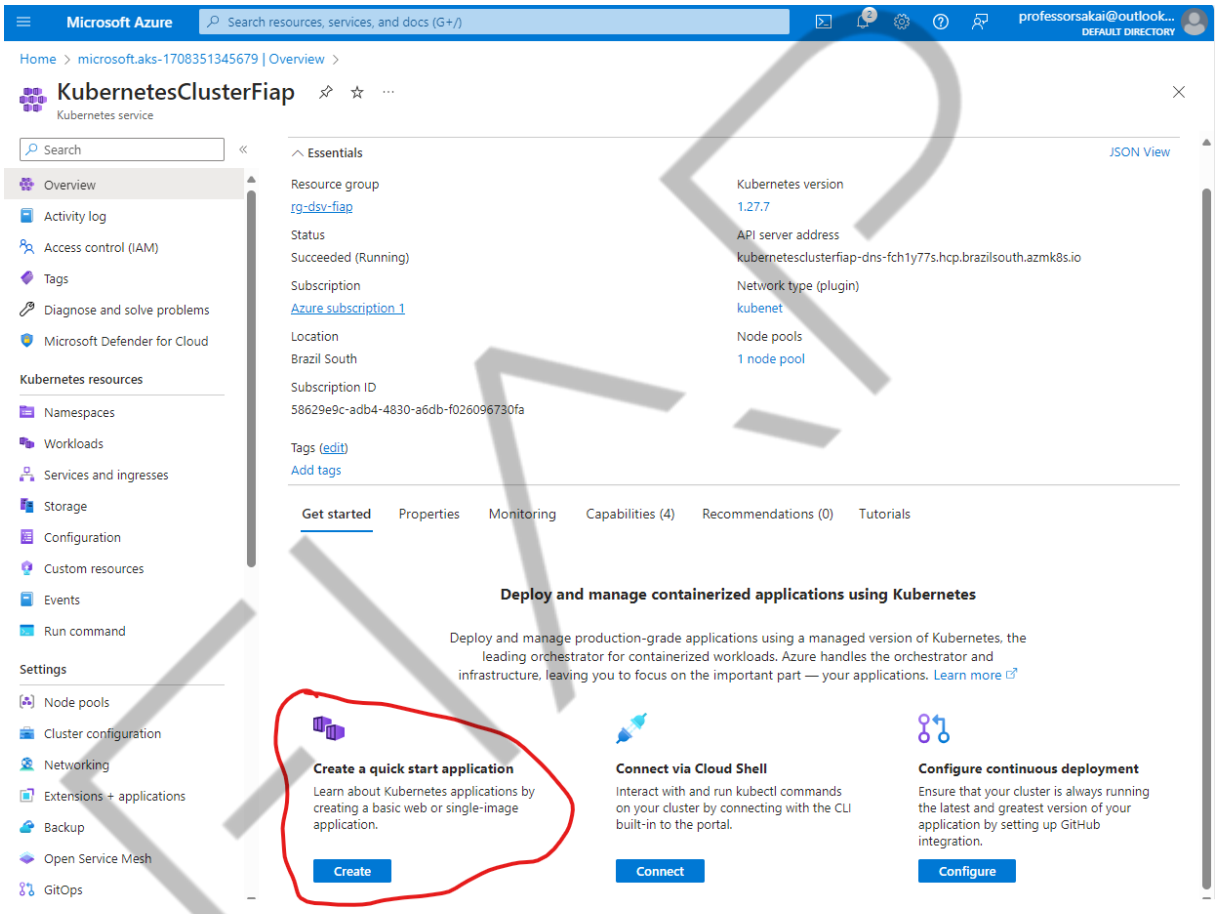

Figura 27 – Criando uma aplicação (1)
Fonte: Elaborado pelo autor (2024)

Na próxima tela, escolher a criação de uma aplicação web básica, conforme Figura 28

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

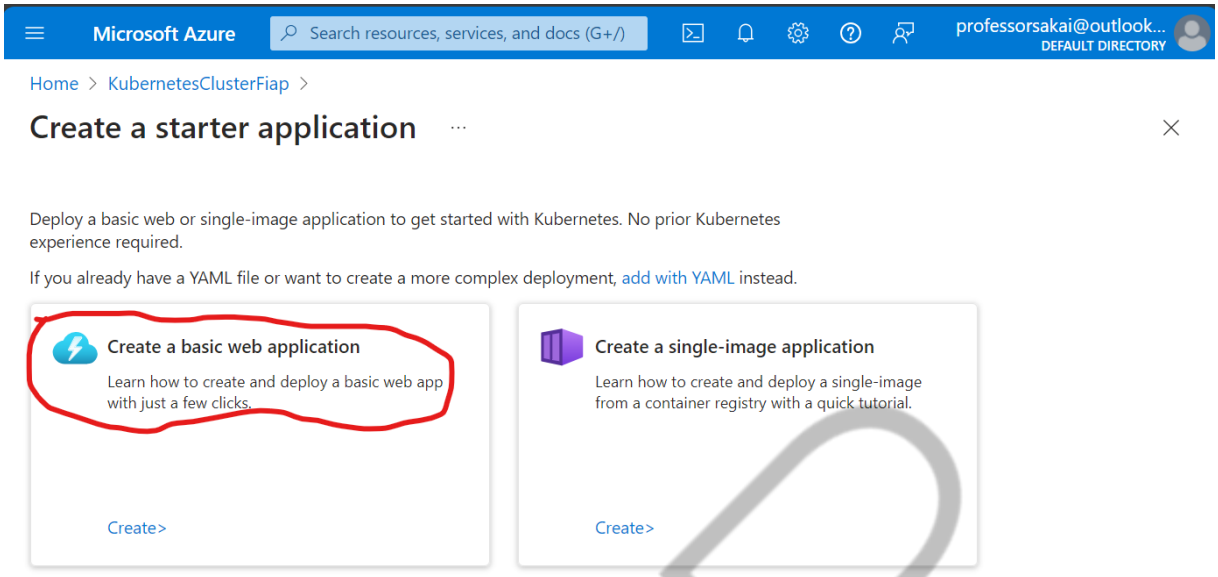

Figura 28 - Criando uma aplicação (2)
Fonte: Elaborado pelo autor (2024)

Na tela seguinte, clique em “next” reveja a configuração YAML e clique em “Deploy”, conforme Figura 29 e Figura 30.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

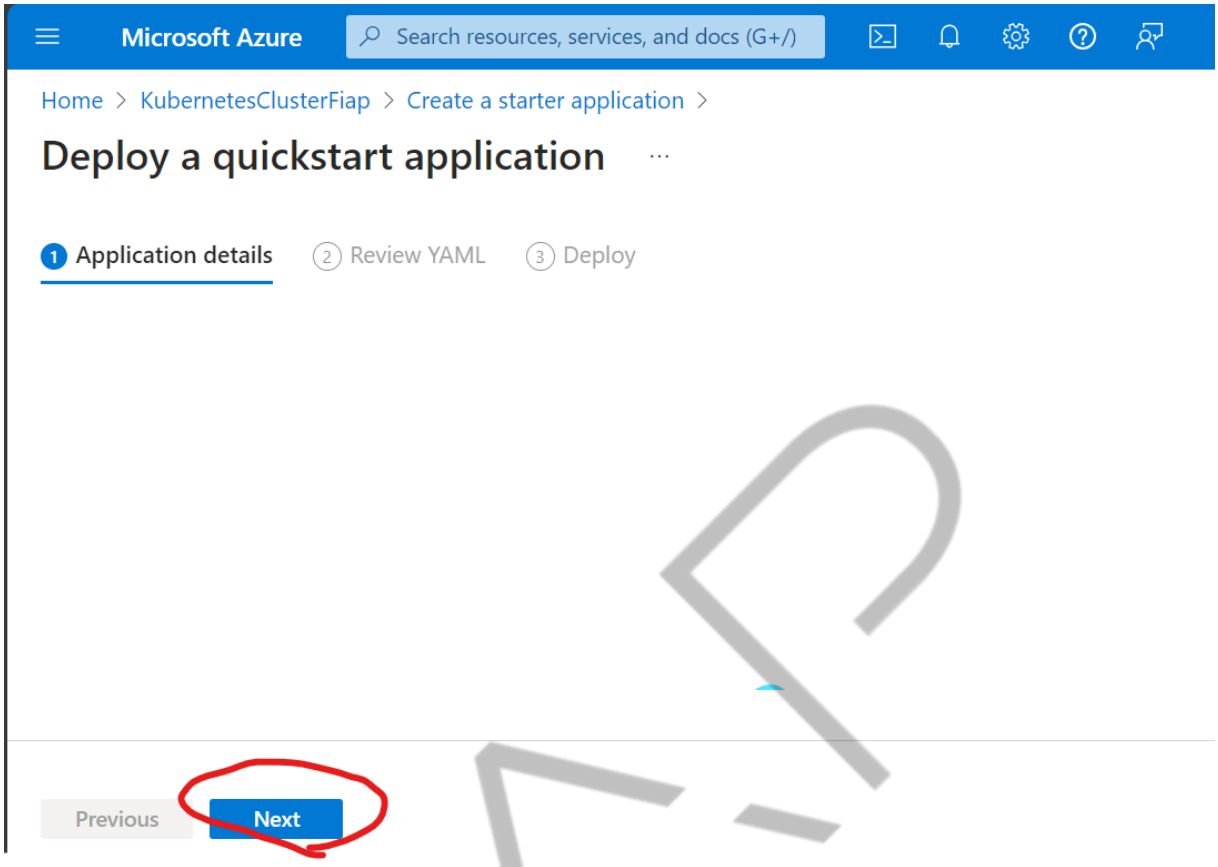

Figura 29 - Criando uma aplicação (3)
Fonte: Elaborado pelo autor (2024)

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

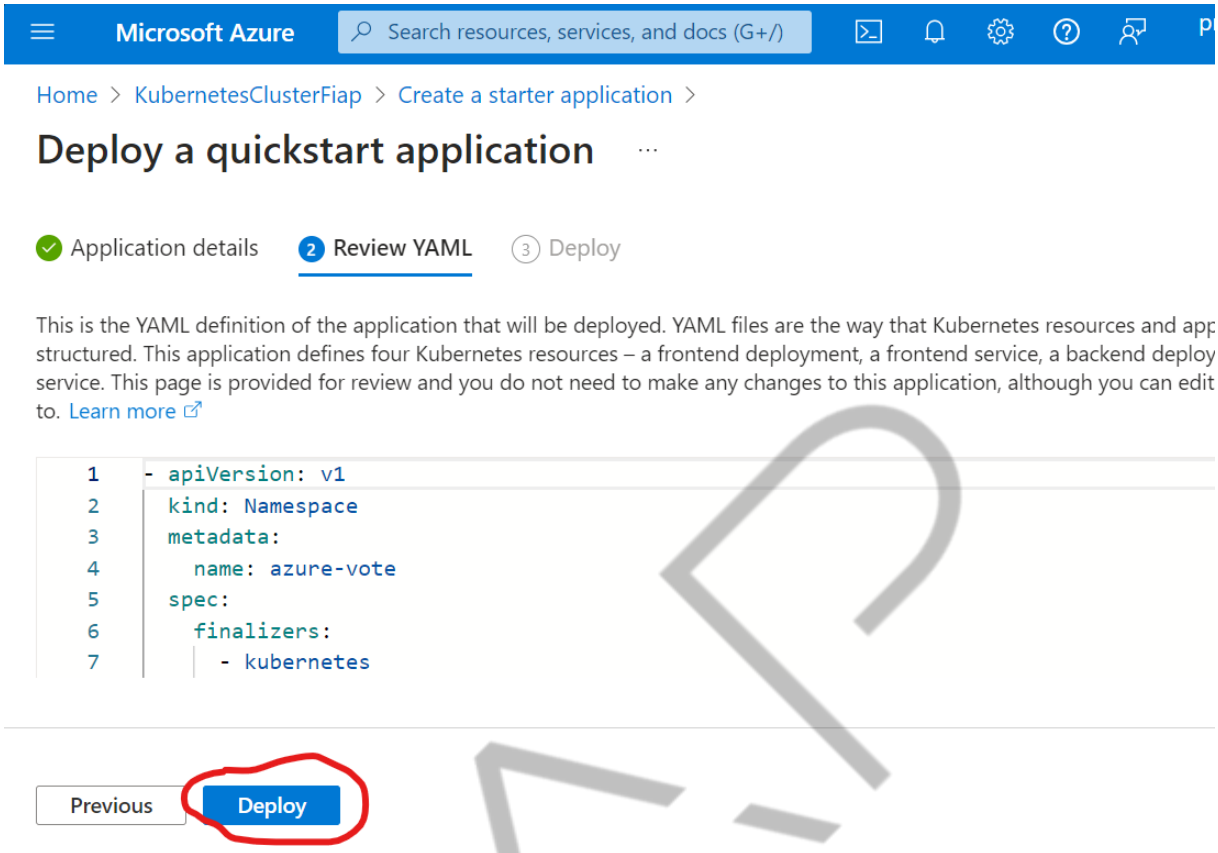

Figura 30 - Criando uma aplicação (4)
Fonte: Elaborado pelo autor (2024)

Aguardar a implantação (deploy) da aplicação em contêiner e depois acessar a aplicação clicando “View Application” conforme Figura 31

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

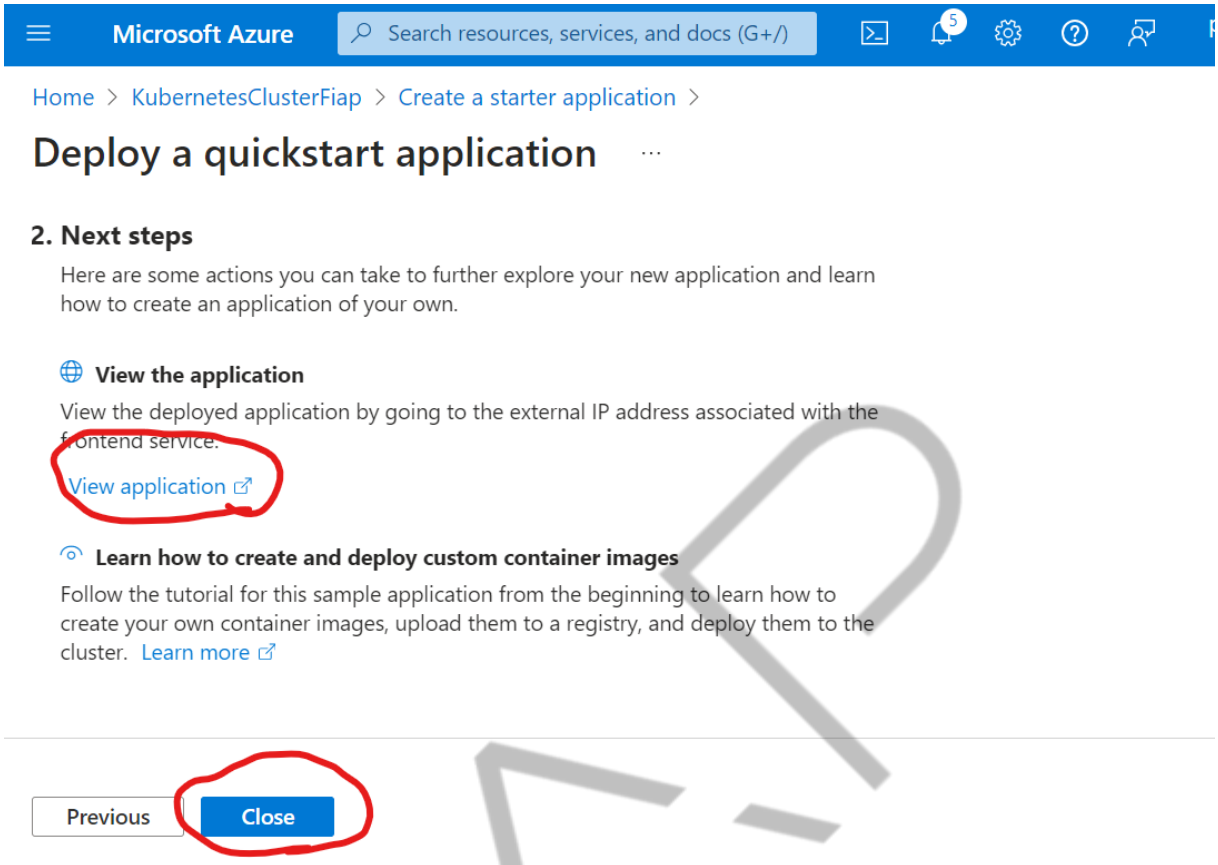

Figura 31 - Acessando a aplicação
Fonte: Elaborado pelo autor (2024)

A Figura 32 mostra a aplicação de exemplo que foi implantada no Kubernetes criado nesse hands-on. É uma aplicação simples que registra os votos em gatos ou cachorros. Faça um teste, envie o endereço (IP) da aplicação para seus colegas e veja os votos atualizando (F5) a página.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

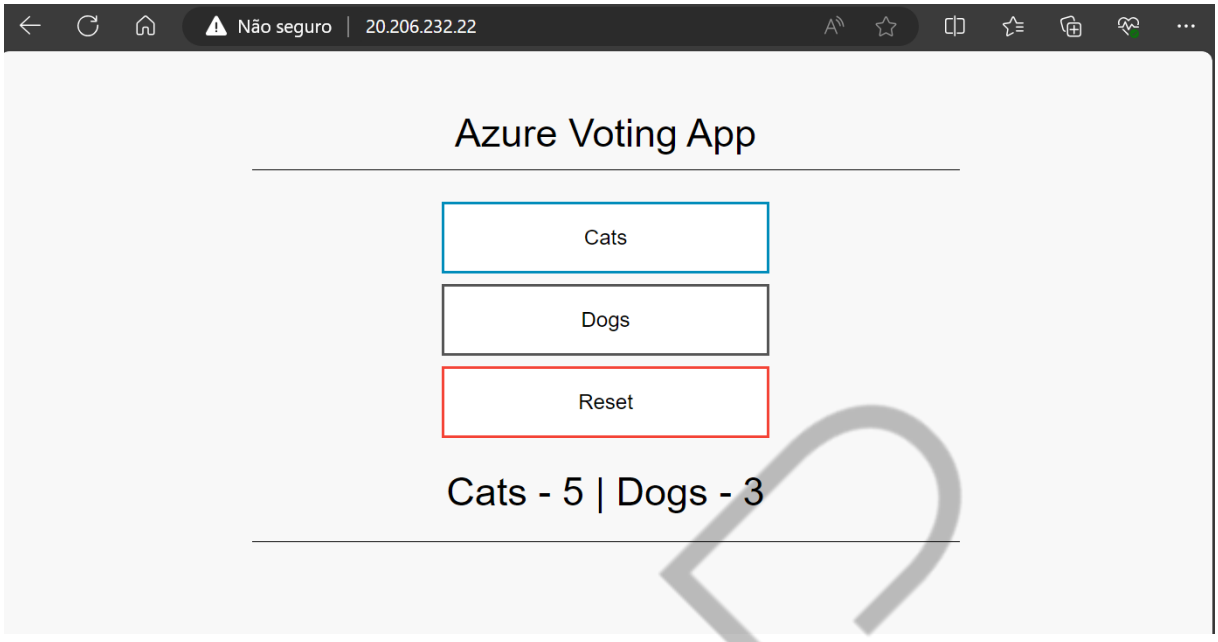

Figura 32 - Aplicação Web
Fonte: Elaborado pelo autor (2024)

Após o término desse hands-on é importante remover os recursos criados, nesse caso, o cluster kubernetes. Isso para evitar gastos financeiros ou consumo excessivo dos créditos concedidos pela Microsoft Azure.

Usando o console da Azure (porta) acessar o recurso Cluster Kubernetes criado e na página principal do recurso, clicar em “delete” conforme Figura 33.

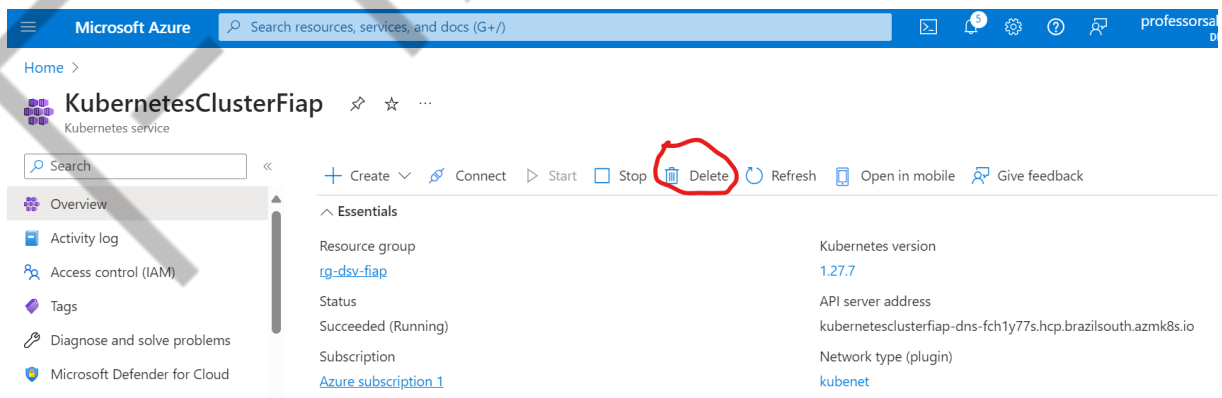

Figura 33 - Removendo o Cluster Kubernetes
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

Clique em “delete” e depois confirme. A figura mostra a tela de opção.

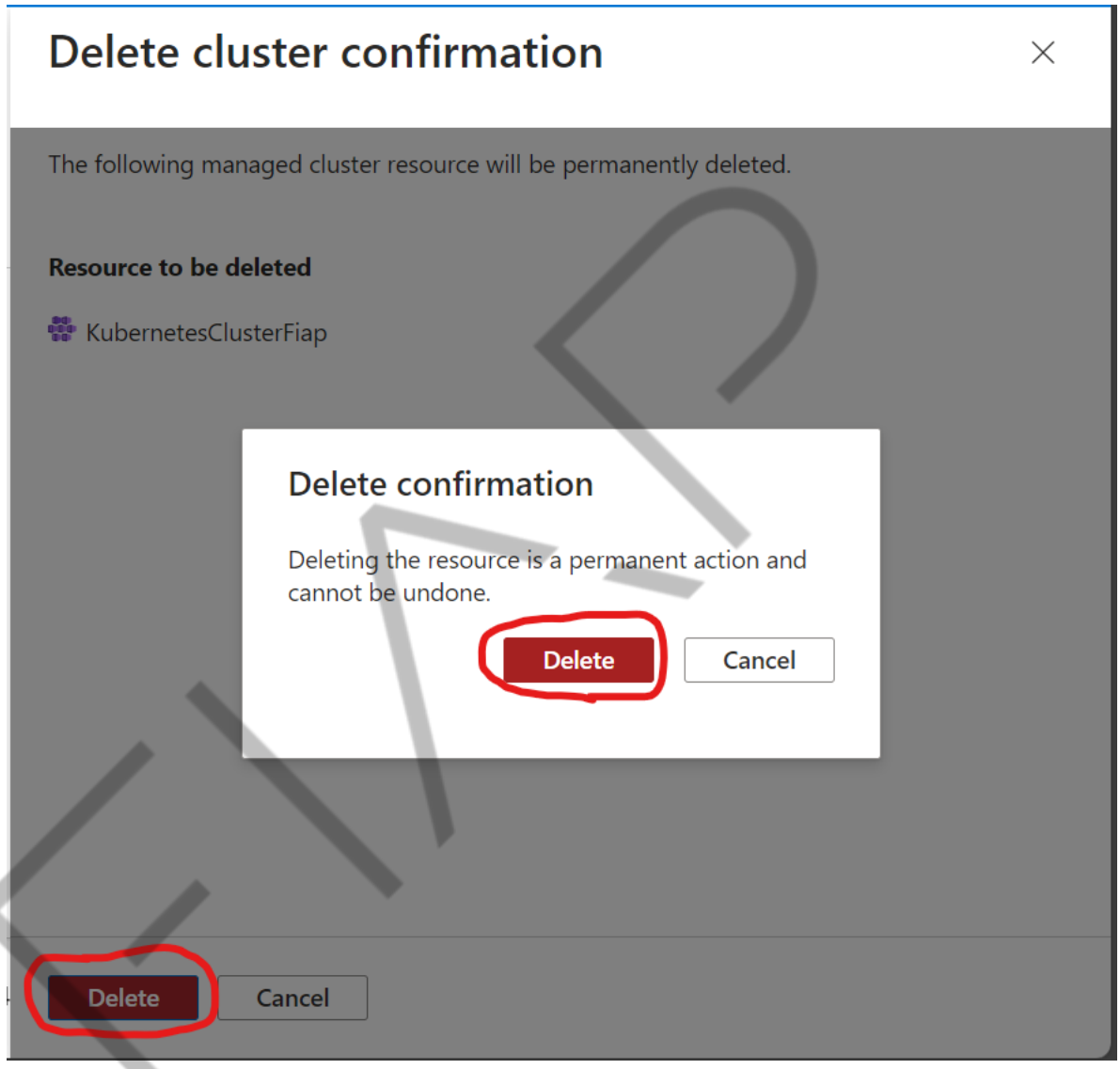

Figura 34 - Removendo o Cluster Kubernetes (2)
Fonte: Elaborado pelo autor (2024)

No exemplo, foram realizados alguns passos para a entrega de uma aplicação simples (Aplicação Web) em Kubernetes. As empresas têm usado cada vez mais a arquitetura *Kubernetes, Docker* e outros recursos na nuvem para disponibilizar desde serviços pontuais, como previsão do tempo, localização geográfica ou processamento de imagens para web, até sistemas completos, como gerenciamento de produtos, estoques e empregados.

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

É possível observar que não foi necessário instalar e configurar um servidor com sistema operacional, compiladores, bibliotecas de funções, serviços de web, etc. Assim, a programação de aplicações torna-se mais produtiva. Ou seja, o time de desenvolvimento fica bem mais focado na solução do problema do que na infraestrutura necessária.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

CONCLUSÃO

A compreensão de como funcionam os containers é absolutamente essencial para todo desenvolvedor, profissional de infraestrutura ou segurança. A cada dia soluções como o Docker tomam maiores partes da infraestrutura das organizações, portanto, compreender bem os padrões abertos de orquestração, como o Kubernetes, será um grande diferencial.

Contudo, este capítulo representa apenas um pontapé inicial em tais conhecimentos e seu objetivo era despertar seu interesse para o assunto. Esperamos que esse objetivo tenha sido cumprido. Até a próxima!

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

REFERÊNCIAS

AMAZON. **AWS documentation website.** [s.d.]. Disponível em: <https://docs.aws.amazon.com>. Acesso em: 11 set. 2019.2 jun. 2021.

CACCIATORE, K.; DAKE, S.; OTTO, A.; *et al.* Exploring opportunities: containers and OpenStack. technical report. **Openstack.org**, 2015. Disponível em: <https://object-storage-ca-ymq-1.vexxhost.net/swift/v1/6e4619c416ff4bd19e1c087f27a43eea/www-assets-prod/pdf-downloads/Containers-and-OpenStack.pdf>. Acesso em: 2 jun. 2021.11 set. 2019.

DOCKER. **Docker website.** [s.d.]. Disponível em: <https://www.docker.com/>. Acesso em: 2 jun. 2021.11 set. 2019.

KUBERNETS. **Kubernets web site.** [s.d.]. Disponível em: <https://kubernetes.io/>. Acesso em: 2 jun. 2021.11 set. 2019.

OPENSTACK. **Kolla documentation website**. 2013. Disponível em: <https://docs.openstack.org/kolla/4.0.0/>. Acesso em: 11 set. 2019.2 jun. 2021.

OPENSTACK. **Magnum user guide website.** 2020. Disponível em: <https://docs.openstack.org/magnum/latest/user/>. Acesso em: 2 jun. 2021.11 set. 2019.

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com

LISTA DE ABREVIATURAS

API	Application Programming Interface
BPM	Business Process Management
EAI	Enterprise Application Integration
HTTP	Hiper Text Transfer Protocol
IDE	Integrated Development Environment
Java EE	Java Enterprise Edition
RPC	Remote Procedure Call
SOA	Service Oriented Architecture
SOAP	Simple Object Access Protocol
TI	Tecnologia da Informação
WSDL	Web Service Description Language
XML	Extensible Markup Language
URL	Uniform Resource Locators
URI	Uniform Resource Identifiers
DDD	Domain-Driven Design
P&D	Pesquisa e Desenvolvimento
VM	Virtual Machine

FIAP

PDF exclusivo para Marcos Antonio Vieira de Almeida - rm559875
marcosdaguealmeida@outlook.com